QuestDB Cookbook

SQL Recipes for Time-Series Data

January 2026

Contents

Introduction . . . 3
Finance . . . 5
- Calculate compound interest . . . 6
- Cumulative product for random walk . . . 8
- Volume weighted average price (VWAP) . . . 10
- Bollinger bands . . . 12
- TICK and TRIN indicators . . . 15
- Aggressor volume imbalance . . . 17
- Volume profile . . . 18
- Volume spike detection . . . 19
- Rolling standard deviation . . . 20

Time Series . . . 21
- Force a designated timestamp . . . 22
- Get latest N records per partition . . . 24
- Calculate sessions and elapsed time . . . 28
- Query last N minutes of activity . . . 33
- Filter data by week number . . . 34
- Distribute discrete values across time intervals . . . 35
- Query with epoch timestamps . . . 37
- Right interval bound with SAMPLE BY . . . 38
- Remove outliers from candle data . . . 39
- Fill missing intervals with value from another column . . . 41
- FILL PREV with historical values . . . 43
- FILL on keyed queries with arbitrary intervals . . . 44
- Join strategies for sparse sensor data . . . 45

Advanced Queries . . . 48
- Access rows before and after current row . . . 49
- Find local minimum and maximum . . . 50
- Top N plus others row . . . 51
- Pivot with “Others” column . . . 55
- Unpivoting query results . . . 57
- Sankey and funnel diagrams . . . 60
- Multiple conditional aggregates . . . 62
- General and sampled aggregates . . . 64
- Consistent histogram buckets . . . 65
- Create arrays from string literals . . . 67

Collect OPC-UA data with Telegraf in dense format . . . 69
Grafana . . . 74
- Query multiple tables dynamically in Grafana . . . 75
- Configure read-only user for Grafana . . . 78
- Grafana variable dropdown with name and value . . . 80
- Overlay two time series with time shift . . . 84

Configure TLS certificate authorities . . . 87
PHP . . . 89
Insert data from PHP using ILP . . . 90
Ruby . . . 96
Insert data from Ruby using ILP . . . 97
C++ . . . 103
Handle missing columns in C++ client . . . 104
Configure QuestDB with Docker Compose . . . 107
Store QuestDB metrics in QuestDB . . . 109
Import CSV with millisecond timestamps . . . 110
TLS with PgBouncer for QuestDB . . . 111
Copy data between QuestDB instances . . . 112
Query performance histogram . . . 113
Optimize disk and memory usage with many tables . . . 115
Check transaction applied after ingestion . . . 116
Show parameters with non-default values . . . 117
Demo Data Schema . . . 119

Introduction

The Cookbook is a collection of **short, actionable recipes** that demonstrate how to accomplish specific tasks with QuestDB. Each recipe follows a problem-solution-result format, making it easy to find and apply solutions quickly.

What is the cookbook?

Unlike comprehensive reference documentation, the Cookbook focuses on practical examples for:

- **Common SQL patterns** - Window functions, pivoting, time-series aggregations
- **Programmatic integration** - Language-specific client examples
- **Operations** - Deployment and configuration tasks

Each recipe provides a focused solution to a specific problem, with working code examples and expected results.

Structure

The Cookbook is organized into three main sections:

- **SQL Recipes** - Common SQL patterns, window functions, and time-series queries
- **Programmatic** - Language-specific client examples and integration patterns
- **Operations** - Deployment, configuration, and operational tasks

Running the examples

Most recipes run directly on our live demo instance at demo.questdb.com without any local setup. Queries that can be executed on the demo site are marked with a direct link to run them.

For recipes that require write operations or specific configuration, the recipe will indicate what setup is needed.

The demo instance contains live FX market data with tables for core prices and order book snapshots. See the Demo Data Schema page for details about available tables and their structure.

Using the cookbook

Each recipe follows a consistent format:

1. **Problem statement** - What you're trying to accomplish
2. **Solution** - Code example with explanation
3. **Results** - Expected output or verification
4. **Additional context** - Tips, variations, or related documentation links

Start by browsing the SQL Recipes section for common patterns, or jump directly to the recipe that matches your needs.

SQL Recipes

Finance

Calculate compound interest

Calculate compound interest over multiple periods using SQL, where each period's interest is calculated on the previous period's ending balance. This is useful for financial modeling, investment projections, and interest calculations.

> **INFO: Generated Data**
>
> This query uses generated data from `long_sequence()` to create a time series of years, so it can run directly on the demo instance without requiring any existing tables.

Problem: Need year-by-year growth You want to calculate compound interest over 5 years, starting with an initial principal of 1000, with an annual interest rate of 0.1 (10%). Each year's interest should be calculated on the previous year's ending balance.

Solution: Use POWER function with window functions The compound interest formula is `principal * (1 + rate)^periods`. Use `POWER()` to calculate the exponential part:

Calculate compound interest over 5 years

```
WITH
year_series AS (
    DECLARE @year:=2000,
            @rate := 0.1,
            @principal := 1000.0
    SELECT @year as start_year, @year + (x - 1) AS timestamp,
    @rate AS interest_rate, @principal as initial_principal
    FROM long_sequence(5) -- number of years
),
compounded_values AS (
    SELECT
        timestamp,
        initial_principal,
        interest_rate,
        initial_principal *
            POWER(
                  1 + interest_rate,
                  timestamp - start_year + 1
                  ) AS compounding
    FROM
        year_series
), compounding_year_before AS (
SELECT
    timestamp,
    initial_principal,
    interest_rate,
    LAG(cv.compounding) OVER (ORDER BY timestamp) AS year_principal,
    cv.compounding as compounding_amount
FROM
    compounded_values cv
ORDER BY
    timestamp
    )
select timestamp, initial_principal, interest_rate,
coalesce(year_principal, initial_principal) as year_principal,
compounding_amount
from compounding_year_before;
```

Run on demo.questdb.com
Results:

timestamp	initial_principal	interest_rate	year_principal	compounding_amount
2000	1000.0	0.1	1000.0	1100.0
2001	1000.0	0.1	1100.0	1210.0
2002	1000.0	0.1	1210.0	1331.0
2003	1000.0	0.1	1331.0	1464.1
2004	1000.0	0.1	1464.1	1610.51

Each row shows how the principal grows year over year, with interest compounding on the previous year's ending balance.

How it works The query uses a multi-step CTE approach:

1. **Generate year series**: Use `long_sequence(5)` to create 5 rows representing years 2000-2004
2. **Calculate compound amount**: Use `POWER(1 + interest_rate, years)` to compute the ending balance for each year
3. **Get previous year's balance**: Use `LAG()` to access the previous row's compounding amount
4. **Handle first year**: Use `COALESCE()` to show the initial principal for the first year

TIP: For more complex scenarios like monthly or quarterly compounding, adjust the time period generation and the exponent in the POWER function accordingly.

View this recipe online

Cumulative product for random walk

Calculate the cumulative product of daily returns to simulate a stock's price path (random walk). This is useful for financial modeling, backtesting trading strategies, and portfolio analysis where you need to compound returns over time.

Problem: Compound daily returns You have a table with daily returns for a stock and want to calculate the cumulative price starting from an initial value (e.g., $100). Each day's price is calculated by multiplying the previous price by `(1 + return)`.

For example, with these daily returns:

Date	Daily Return (%)
2024-09-05	2.00
2024-09-06	-1.00
2024-09-07	1.50
2024-09-08	-3.00

You want to calculate:

Date	Daily Return (%)	Stock Price
2024-09-05	2.00	102.00
2024-09-06	-1.00	100.98
2024-09-07	1.50	102.49
2024-09-08	-3.00	99.42

Solution: Use logarithms to convert multiplication to addition Use the mathematical identity: **exp(sum(ln(x))) = product(x)**

This converts the cumulative product into a cumulative sum, which window functions handle naturally:

```
WITH ln_values AS (
    SELECT
        date,
        return,
        SUM(ln(1 + return)) OVER (ORDER BY date) AS ln_value
    FROM daily_returns
)
SELECT
    date,
    return,
    100 * exp(ln_value) AS stock_price
FROM ln_values;
```

This query: 1. Calculates `ln(1 + return)` for each day 2. Uses a cumulative `SUM` window function to add up the logarithms 3. Applies `exp()` to convert back to the product

How it works The mathematical identity used here is:

$$\text{product}(1 + r_1, 1 + r_2, \ldots, 1 + r_n) = \exp(\text{sum}(\ln(1 + r_1), \ln(1 + r_2), \ldots, \ln(1 + r_n)))$$

Breaking it down: - `ln(1 + return)` converts each multiplicative factor to an additive one - `SUM(...) OVER (ORDER BY date)` creates a cumulative sum - `exp(ln_value)` converts the cumulative sum back to a cumulative product - Multiply by 100 to apply the starting price of $100

Adapting to your data You can easily modify this pattern:

Different starting price:

```
SELECT date, return, 1000 * exp(ln_value) AS stock_price  -- Start at $1000
FROM ln_values;
```

Different time granularity:

```
-- For hourly returns
WITH ln_values AS (
    SELECT
        timestamp,
        return,
        SUM(ln(1 + return)) OVER (ORDER BY timestamp) AS ln_value
    FROM hourly_returns
)
SELECT timestamp, 100 * exp(ln_value) AS price FROM ln_values;
```

Multiple assets:

```
WITH ln_values AS (
    SELECT
        date,
        symbol,
        return,
        SUM(ln(1 + return)) OVER (PARTITION BY symbol ORDER BY date) AS ln_value
    FROM daily_returns
)
SELECT
    date,
    symbol,
    100 * exp(ln_value) AS stock_price
FROM ln_values;
```

TIP: Use Case: Monte Carlo Simulation

This pattern is essential for Monte Carlo simulations in finance. Generate random returns, apply this cumulative product calculation, and run thousands of iterations to model possible future price paths.

View this recipe online

Volume weighted average price (VWAP)

Calculate the cumulative Volume Weighted Average Price (VWAP) for intraday trading analysis. VWAP is a trading benchmark that represents the average price at which an asset has traded throughout the day, weighted by volume. It's widely used by institutional traders to assess execution quality and identify trend strength.

Problem: Calculate running VWAP You want to calculate the cumulative VWAP for a trading day, where each point shows the average price weighted by volume from market open until that moment. This helps traders determine if current prices are above or below the day's volume-weighted average.

Solution: Use typical price from OHLC data The industry standard for VWAP uses the **typical price** formula from OHLC (Open, High, Low, Close) candles:

```
Typical Price = (High + Low + Close) / 3
VWAP = Σ(Typical Price × Volume) / Σ(Volume)
```

This approximation is used because most trading platforms work with OHLC data rather than tick-level trades. We use the `fx_trades_ohlc_1m` materialized view which provides 1-minute candles:

Calculate cumulative VWAP

```
WITH sampled AS (
  SELECT
    timestamp, symbol,
    total_volume,
    ((high + low + close) / 3) * total_volume AS traded_value
  FROM fx_trades_ohlc_1m
  WHERE timestamp IN yesterday() AND symbol = 'EURUSD'
),
cumulative AS (
  SELECT
    timestamp, symbol,
    SUM(traded_value) OVER (ORDER BY timestamp) AS cumulative_value,
    SUM(total_volume) OVER (ORDER BY timestamp) AS cumulative_volume
  FROM sampled
)
SELECT timestamp, symbol, cumulative_value / cumulative_volume AS vwap
FROM cumulative;
```

Run on demo.questdb.com

This query: 1. Reads 1-minute OHLC candles and calculates typical price × volume for each candle 2. Uses window functions to compute running totals of both traded value and volume 3. Divides cumulative traded value by cumulative volume to get VWAP at each timestamp

How it works The key insight is using `SUM(...) OVER (ORDER BY timestamp)` to create running totals: - `cumulative_value`: Running sum of (typical price × volume) from market open - `cumulative_volume`: Running sum of volume from market open - Final VWAP: Dividing these cumulative values gives the volume-weighted average at each point

When using `SUM() OVER (ORDER BY timestamp)` without specifying a frame clause, QuestDB defaults to summing from the first row to the current row, which is exactly what we need for cumulative VWAP.

Multiple symbols To calculate VWAP for multiple symbols simultaneously, add `PARTITION BY symbol` to the window functions:

VWAP for multiple symbols

```
WITH sampled AS (
  SELECT
    timestamp, symbol,
    total_volume,
    ((high + low + close) / 3) * total_volume AS traded_value
  FROM fx_trades_ohlc_1m
  WHERE timestamp IN yesterday()
    AND symbol IN ('EURUSD', 'GBPUSD', 'USDJPY')
),
cumulative AS (
  SELECT
    timestamp, symbol,
    SUM(traded_value) OVER (PARTITION BY symbol ORDER BY timestamp) AS cumulative_value,
    SUM(total_volume) OVER (PARTITION BY symbol ORDER BY timestamp) AS cumulative_volume
  FROM sampled
)
SELECT timestamp, symbol, cumulative_value / cumulative_volume AS vwap
FROM cumulative;
```

Run on demo.questdb.com

The `PARTITION BY symbol` ensures each symbol's VWAP is calculated independently, resetting the cumulative sums for each symbol.

Different time ranges

```
-- Current trading day
WHERE timestamp IN today()

-- Specific date
WHERE timestamp IN '2026-01-12'

-- Last hour
WHERE timestamp >= dateadd('h', -1, now())
```

TIP: Trading use cases

- **Execution quality**: Institutional traders compare their execution prices against VWAP to assess trade quality
- **Trend identification**: Price consistently above VWAP suggests bullish momentum; below suggests bearish
- **Support/resistance**: VWAP often acts as dynamic support or resistance during the trading day
- **Mean reversion**: Traders use deviations from VWAP to identify potential reversal points

View this recipe online

Bollinger bands

Calculate Bollinger Bands for volatility analysis and mean reversion trading. Bollinger Bands consist of a moving average with upper and lower bands set at a specified number of standard deviations above and below it. They help identify overbought/oversold conditions and measure market volatility.

Solution: Calculate variance using window functions Since standard deviation is the square root of variance, and variance is the average of squared differences from the mean, we can calculate everything in SQL using window functions. This query will compute Bollinger Bands with a 20-period simple moving average (SMA) and bands at ±2 standard deviations:

Calculate Bollinger Bands with 20-period SMA

```
WITH OHLC AS (
  SELECT
    timestamp, symbol,
      first(price) AS open,
      max(price) as high,
      min(price) as low,
      last(price) AS close,
      sum(quantity) AS volume
 FROM fx_trades
 WHERE symbol = 'EURUSD' AND timestamp IN yesterday()
 SAMPLE BY 15m
), stats AS (
  SELECT
    timestamp,
    close,
    AVG(close) OVER (
      ORDER BY timestamp
      ROWS 19 PRECEDING
    ) AS sma20,
    AVG(close * close) OVER (
      ORDER BY timestamp
      ROWS 19 PRECEDING
    ) AS avg_close_sq
  FROM OHLC
)
SELECT
  timestamp,
  close,
  sma20,
  sqrt(avg_close_sq - (sma20 * sma20)) as stdev20,
  sma20 + 2 * sqrt(avg_close_sq - (sma20 * sma20)) as upper_band,
  sma20 - 2 * sqrt(avg_close_sq - (sma20 * sma20)) as lower_band
FROM stats
ORDER BY timestamp;
```

Run on demo.questdb.com

This query: 1. Aggregates trades into 15-minute OHLC candles 2. Calculates a 20-period simple moving average of closing prices 3. Calculates the average of squared closing prices over the same 20-period window 4. Computes standard deviation using the mathematical identity: $\sigma = \sqrt{(E[X^2] - E[X]^2)}$ 5. Adds/subtracts 2× standard deviation to create upper and lower bands

How it works The core of the Bollinger Bands calculation is the rolling standard deviation. Please check our rolling standard deviation recipe in the cookbook for an explanation about the mathematical formula.

Adapting the parameters **Different period lengths:**

```
-- 10-period Bollinger Bands (change 19 to 9)
AVG(close) OVER (ORDER BY timestamp ROWS 9 PRECEDING) AS sma10,
AVG(close * close) OVER (ORDER BY timestamp ROWS 9 PRECEDING) AS avg_close_sq
```

Different band multipliers:

```
-- 1 standard deviation bands (tighter)
sma20 + 1 * sqrt(avg_close_sq - (sma20 * sma20)) as upper_band,
sma20 - 1 * sqrt(avg_close_sq - (sma20 * sma20)) as lower_band

-- 3 standard deviation bands (wider)
sma20 + 3 * sqrt(avg_close_sq - (sma20 * sma20)) as upper_band,
sma20 - 3 * sqrt(avg_close_sq - (sma20 * sma20)) as lower_band
```

Different time intervals:

```
-- 5-minute candles
SAMPLE BY 5m

-- 1-hour candles
SAMPLE BY 1h
```

Multiple symbols: Bollinger Bands for multiple symbols

```
WITH OHLC AS (
  SELECT
    timestamp, symbol,
      first(price) AS open,
      last(price) AS close,
      sum(quantity) AS volume
 FROM fx_trades
 WHERE symbol IN ('EURUSD', 'GBPUSD')
   AND timestamp IN yesterday()
 SAMPLE BY 15m
), stats AS (
  SELECT
    timestamp,
    symbol,
    close,
    AVG(close) OVER (
      PARTITION BY symbol
      ORDER BY timestamp
      ROWS 19 PRECEDING
    ) AS sma20,
    AVG(close * close) OVER (
      PARTITION BY symbol
      ORDER BY timestamp
      ROWS 19 PRECEDING
    ) AS avg_close_sq
  FROM OHLC
)
```

```
SELECT
  timestamp,
  symbol,
  close,
  sma20,
  sma20 + 2 * sqrt(avg_close_sq - (sma20 * sma20)) as upper_band,
  sma20 - 2 * sqrt(avg_close_sq - (sma20 * sma20)) as lower_band
FROM stats
ORDER BY symbol, timestamp;
```

Run on demo.questdb.com

Note the addition of `PARTITION BY symbol` to calculate separate Bollinger Bands for each symbol.

View this recipe online

TICK and TRIN indicators

Calculate TICK and TRIN (Trading Index, also known as the ARMS Index) to measure market breadth. These indicators classify each time period as advancing or declining based on price movement.

Problem: Measure market breadth by price direction You want to calculate TICK and TRIN indicators using traditional definitions: - **Uptick**: Current price > previous price - **Downtick**: Current price < previous price - **TICK** = upticks - downticks - **TRIN** = (upticks / downticks) / (uptick_volume / downtick_volume)

Solution: Use LAG to compare consecutive prices

Per-symbol TICK and TRIN Calculate separate indicators for each currency pair:

TICK and TRIN per symbol

```
WITH candles AS (
  SELECT timestamp, symbol, last(price) AS close, sum(quantity) AS total_volume
  FROM fx_trades
  WHERE timestamp IN yesterday()
    AND symbol IN ('EURUSD', 'GBPUSD', 'USDJPY')
  SAMPLE BY 10m
),
prev_prices AS (
  SELECT timestamp, symbol, close, total_volume,
    LAG(close) OVER (PARTITION BY symbol ORDER BY timestamp) AS prev_close
  FROM candles
),
classified AS (
  SELECT *,
    CASE WHEN close > prev_close THEN 1 ELSE 0 END AS is_uptick,
    CASE WHEN close < prev_close THEN 1 ELSE 0 END AS is_downtick,
    CASE WHEN close > prev_close THEN total_volume ELSE 0 END AS uptick_vol,
    CASE WHEN close < prev_close THEN total_volume ELSE 0 END AS downtick_vol
  FROM prev_prices
  WHERE prev_close IS NOT NULL
),
aggregated AS (
  SELECT symbol,
    SUM(is_uptick) AS upticks,
    SUM(is_downtick) AS downticks,
    SUM(is_uptick) - SUM(is_downtick) AS tick,
    SUM(uptick_vol) AS uptick_vol,
    SUM(downtick_vol) AS downtick_vol
  FROM classified
)
SELECT symbol,
  upticks,
  downticks,
  tick,
  upticks::double / downticks AS advance_decline_ratio,
  uptick_vol::double / downtick_vol AS upside_downside_ratio,
  (upticks::double / downticks) / (uptick_vol::double / downtick_vol) AS trin
FROM aggregated;
```

Run on demo.questdb.com

Market-wide TICK and TRIN Aggregate across all symbols for a single market breadth reading:

Market-wide TICK and TRIN

```
WITH candles AS (
  SELECT timestamp, symbol, last(price) AS close, sum(quantity) AS total_volume
  FROM fx_trades
  WHERE timestamp IN yesterday()
    AND symbol IN ('EURUSD', 'GBPUSD', 'USDJPY')
  SAMPLE BY 10m
),
prev_prices AS (
  SELECT timestamp, symbol, close, total_volume,
    LAG(close) OVER (PARTITION BY symbol ORDER BY timestamp) AS prev_close
  FROM candles
),
classified AS (
  SELECT *,
    CASE WHEN close > prev_close THEN 1 ELSE 0 END AS is_uptick,
    CASE WHEN close < prev_close THEN 1 ELSE 0 END AS is_downtick,
    CASE WHEN close > prev_close THEN total_volume ELSE 0 END AS uptick_vol,
    CASE WHEN close < prev_close THEN total_volume ELSE 0 END AS downtick_vol
  FROM prev_prices
  WHERE prev_close IS NOT NULL
),
aggregated AS (
  SELECT
    SUM(is_uptick) AS upticks,
    SUM(is_downtick) AS downticks,
    SUM(is_uptick) - SUM(is_downtick) AS tick,
    SUM(uptick_vol) AS uptick_vol,
    SUM(downtick_vol) AS downtick_vol
  FROM classified
)
SELECT
  upticks,
  downticks,
  tick,
  upticks::double / downticks AS advance_decline_ratio,
  uptick_vol::double / downtick_vol AS upside_downside_ratio,
  (upticks::double / downticks) / (uptick_vol::double / downtick_vol) AS trin
FROM aggregated;
```

Run on demo.questdb.com

Interpreting the indicators **TICK:** - **Positive**: More upticks than downticks (bullish) - **Negative**: More downticks than upticks (bearish) - **Near zero**: Balanced market

TRIN (ARMS Index): - **< 1.0**: Volume favoring advances (bullish) - **> 1.0**: Volume favoring declines (bearish) - **= 1.0**: Neutral

> **NOTE: TRIN limitations**
>
> TRIN can produce counterintuitive results. For example, if advances outnumber declines 2:1 and advancing volume also leads 2:1, TRIN equals 1.0 (neutral) despite bullish conditions. The query includes separate **advance_decline_ratio** and **upside_downside_ratio** columns to help identify such cases.

View this recipe online

Aggressor volume imbalance

Calculate the imbalance between buy and sell aggressor volume to analyze order flow. The aggressor is the party that initiated the trade by crossing the spread.

Problem: Measure order flow imbalance You have trade data with a `side` column indicating the aggressor (buyer or seller), and want to measure the imbalance between buying and selling pressure.

Solution: Aggregate by side and calculate ratios **Aggressor volume imbalance per symbol**

```
WITH volumes AS (
  SELECT
    symbol,
    SUM(CASE WHEN side = 'buy' THEN amount ELSE 0 END) AS buy_volume,
    SUM(CASE WHEN side = 'sell' THEN amount ELSE 0 END) AS sell_volume
  FROM trades
  WHERE timestamp IN yesterday()
    AND symbol IN ('ETH-USDT', 'BTC-USDT', 'ETH-BTC')
)
SELECT
  symbol,
  buy_volume,
  sell_volume,
  ((buy_volume - sell_volume)::double / (buy_volume + sell_volume)) * 100 AS imbalance
FROM volumes;
```

Run on demo.questdb.com

The imbalance ranges from -100% (all sell) to +100% (all buy), with 0% indicating balanced flow.

View this recipe online

Volume profile

Calculate volume profile to show the distribution of trading volume across different price levels.

Solution Group trades into price bins using `FLOOR` and a tick size parameter:

Calculate volume profile with fixed tick size

```
DECLARE @tick_size := 1.0
SELECT
  floor(price / @tick_size) * @tick_size AS price_bin,
  round(SUM(quantity), 2) AS volume
FROM fx_trades
WHERE symbol = 'EURUSD'
  AND timestamp IN today()
ORDER BY price_bin;
```

Run on demo.questdb.com

Since QuestDB does an implicit GROUP BY on all non-aggregated columns, you can omit the explicit GROUP BY clause.

Dynamic tick size For consistent histograms across different price ranges, calculate the tick size dynamically to always produce approximately 50 bins:

Volume profile with dynamic 50-bin distribution

```
WITH raw_data AS (
  SELECT price, quantity
  FROM fx_trades
  WHERE symbol = 'EURUSD' AND timestamp IN today()
),
tick_size AS (
  SELECT (max(price) - min(price)) / 49 as tick_size
  FROM raw_data
)
SELECT
  floor(price / tick_size) * tick_size AS price_bin,
  round(SUM(quantity), 2) AS volume
FROM raw_data CROSS JOIN tick_size
ORDER BY 1;
```

Run on demo.questdb.com

This will produce a histogram with a maximum of 50 buckets. If you have enough price difference between the first and last price for the interval, and if there are enough events with different prices, then you will get the full 50 buckets. If price difference is too small or if there are buckets with no events, then you might get less than 50.

View this recipe online

Volume spike detection

Detect volume spikes by comparing current trading volume against the previous candle's volume.

Problem You have candles aggregated at 30 seconds intervals, and you want to show a flag 'spike' if volume is bigger than twice the latest record for the same symbol. Otherwise it should display 'normal'.

Solution Use the `LAG` window function to retrieve the previous candle's volume, then compare with a `CASE` statement:

Detect volume spikes exceeding 2x previous volume

```
DECLARE
  @anchor_date := timestamp_floor('30s', now()),
  @start_date := dateadd('h', -7, @anchor_date),
  @symbol := 'EURUSD'
WITH candles AS (
  SELECT
    timestamp,
    symbol,
    sum(quantity) AS volume
  FROM fx_trades
  WHERE timestamp >= @start_date
    AND symbol = @symbol
  SAMPLE BY 30s
),
prev_volumes AS (
  SELECT
    timestamp,
    symbol,
    volume,
    LAG(volume) OVER (PARTITION BY symbol ORDER BY timestamp) AS prev_volume
  FROM candles
)
SELECT
  *,
  CASE
    WHEN volume > 2 * prev_volume THEN 'spike'
    ELSE 'normal'
  END AS spike_flag
FROM prev_volumes;
```

Run on demo.questdb.com
View this recipe online

Rolling standard deviation

Calculate rolling standard deviation to measure price volatility over time.

Problem You want to calculate rolling standard deviation.

Solution Use the mathematical identity: `σ = √(E[X²] − E[X]²)`

Compute both `AVG(price)` and `AVG(price * price)` as window functions, then derive the standard deviation:

Calculate rolling standard deviation

```
WITH stats AS (
  SELECT
    timestamp,
    symbol,
    price,
    AVG(price) OVER (PARTITION BY symbol ORDER BY timestamp) AS rolling_avg,
    AVG(price * price) OVER (PARTITION BY symbol ORDER BY timestamp) AS rolling_avg_sq
  FROM fx_trades
  WHERE timestamp IN yesterday() AND symbol = 'EURUSD'
)
SELECT
  timestamp,
  symbol,
  price,
  rolling_avg,
  SQRT(rolling_avg_sq - rolling_avg * rolling_avg) AS rolling_stddev
FROM stats
LIMIT 10;
```

Run on demo.questdb.com

How it works The mathematical relationship used here is:

```
Variance(X) = E[X²] − (E[X])²
StdDev(X) = √(E[X²] − (E[X])²)
```

Where: - `E[X]` is the average (SMA) of prices - `E[X²]` is the average of squared prices - √ is the square root function

View this recipe online

Time Series

Force a designated timestamp

Sometimes you need to force a designated timestamp in your query. This happens when you want to run operations like `SAMPLE BY` with a non-designated timestamp column, or when QuestDB applies certain functions or joins and loses track of the designated timestamp.

Problem: Lost designated timestamp When you run this query on the demo instance, you'll notice the `time` column is not recognized as a designated timestamp because we cast it to a string and back:

Query without designated timestamp

```
SELECT
  TO_TIMESTAMP(timestamp::STRING, 'yyyy-MM-ddTHH:mm:ss.SSSUUUZ') time,
  symbol,
  ecn,
  bid_price
FROM
  core_price
WHERE timestamp IN today()
LIMIT 10;
```

Run on demo.questdb.com

Without a designated timestamp, you cannot use time-series operations like `SAMPLE BY`.

Solution: Use the TIMESTAMP keyword You can force the designated timestamp using the `TIMESTAMP()` keyword, which allows you to run time-series operations:

Force designated timestamp with TIMESTAMP keyword

```
WITH t AS (
  (
    SELECT
      TO_TIMESTAMP(timestamp::STRING, 'yyyy-MM-ddTHH:mm:ss.SSSUUUZ') time,
      symbol,
      ecn,
      bid_price
    FROM
      core_price
    WHERE timestamp >= dateadd('h', -1, now())
    ORDER BY time
  ) TIMESTAMP (time)
)
SELECT * FROM t LATEST BY symbol;
```

Run on demo.questdb.com

The `TIMESTAMP(time)` clause explicitly tells QuestDB which column to use as the designated timestamp, enabling `LATEST BY` and other time-series operations. This query gets the most recent price for each symbol in the last hour.

Common case: UNION queries The designated timestamp is often lost when using `UNION` or `UNION ALL`. This is intentional - QuestDB cannot guarantee that the combined results are in order, and designated timestamps must always be in ascending order.

You can restore the designated timestamp by applying `ORDER BY` and then using `TIMESTAMP()`:

Restore designated timestamp after UNION ALL

```
(
  SELECT * FROM
  (
    SELECT timestamp, symbol FROM core_price WHERE timestamp >= dateadd('m', -1, now())
    UNION ALL
    SELECT timestamp, symbol FROM core_price WHERE timestamp >= dateadd('m', -1, now())
  ) ORDER BY timestamp
)
TIMESTAMP(timestamp)
LIMIT 10;
```

Run on demo.questdb.com

This query combines the last minute of data twice using `UNION ALL`, then restores the designated timestamp.

Querying external Parquet files When querying external parquet files using `read_parquet()`, the result does not have a designated timestamp. You need to force it using `TIMESTAMP()` to enable time-series operations like `SAMPLE BY`:

Query parquet file with designated timestamp

```
SELECT timestamp, avg(price)
FROM ((read_parquet('trades.parquet') ORDER BY timestamp) TIMESTAMP(timestamp))
SAMPLE BY 1m;
```

Run on demo.questdb.com

This query reads from a parquet file, applies ordering, forces the designated timestamp, and then performs time-series aggregation.

> **WARNING: Order is Required**
>
> The `TIMESTAMP()` keyword requires that the data is already sorted by the timestamp column. If the data is not in order, the query will fail. Always include `ORDER BY` before applying `TIMESTAMP()`.

View this recipe online

Get latest N records per partition

Retrieve the most recent N rows for each distinct partition value (e.g., latest 5 trades per symbol, last 10 readings per sensor). While `LATEST ON` returns only the single most recent row per partition, this pattern extends it to get multiple recent rows per partition.

Problem: Need multiple recent rows per group You want to get the latest N rows for each distinct value in a column. For example: - Latest 5 trades for each trading symbol - Last 10 sensor readings per device - Most recent 3 log entries per service

`LATEST ON` only returns one row per partition:

LATEST ON returns only 1 row per symbol

```
SELECT * FROM trades
WHERE timestamp in today()
LATEST ON timestamp PARTITION BY symbol;
```

Run on demo.questdb.com

But you need multiple rows per symbol.

Solution: Use ROW_NUMBER() window function Use `row_number()` to rank rows within each partition, then filter to keep only the top N:

Get latest 5 trades for each symbol

```
WITH ranked AS (
  SELECT
    *,
    row_number() OVER (PARTITION BY symbol ORDER BY timestamp DESC) as rn
  FROM trades
  WHERE timestamp in today()
)
SELECT timestamp, symbol, side, price, amount
FROM ranked
WHERE rn <= 5
ORDER BY symbol, timestamp DESC;
```

Run on demo.questdb.com

This returns up to 5 most recent trades for each symbol from the last day.

How it works The query uses a two-step approach:

1. **Ranking step (CTE):**
   - `row_number() OVER (...)`: Assigns sequential numbers to rows within each partition
   - `PARTITION BY symbol`: Separate ranking for each symbol
   - `ORDER BY timestamp DESC`: Newest rows get lower numbers (1, 2, 3, …)
   - Result: Each row gets a rank within its symbol group
2. **Filtering step (outer query):**
   - `WHERE rn <= 5`: Keep only rows ranked 1-5 (the 5 most recent)
   - `ORDER BY symbol, timestamp DESC`: Sort final results

Understanding row_number() `row_number()` assigns a unique sequential number within each partition:

timestamp	symbol	price	(row number)
10:03:00	BTC-USDT	63000	1 (newest)
10:02:00	BTC-USDT	62900	2
10:01:00	BTC-USDT	62800	3
10:03:30	ETH-USDT	3100	1 (newest)
10:02:30	ETH-USDT	3095	2

With `WHERE rn <= 3`, we keep rows 1-3 for each symbol.

Adapting the query **Different partition columns:**

```
-- Latest 10 per sensor_id
PARTITION BY sensor_id

-- Latest 5 per combination of symbol and exchange
PARTITION BY symbol, exchange

-- Latest N per user_id
PARTITION BY user_id
```

Different sort orders:

```
-- Oldest N rows per partition
ORDER BY timestamp ASC

-- Highest prices first
ORDER BY price DESC

-- Alphabetically
ORDER BY name ASC
```

Dynamic N value: Latest N trades with variable limit

```
DECLARE @limit := 10

WITH ranked AS (
  SELECT *, row_number() OVER (PARTITION BY symbol ORDER BY timestamp DESC) as rn
  FROM trades
  WHERE timestamp >= dateadd('d', -1, now())
)
SELECT * FROM ranked WHERE rn <= @limit;
```

Run on demo.questdb.com

Include additional filtering: Latest 5 buy orders per symbol

```
WITH ranked AS (
  SELECT
    *,
    row_number() OVER (PARTITION BY symbol ORDER BY timestamp DESC) as rn
  FROM trades
  WHERE timestamp in today()
    AND side = 'buy'  -- Additional filter before ranking
```

```
)
SELECT timestamp, symbol, side, price, amount
FROM ranked
WHERE rn <= 5;
```

Run on demo.questdb.com

Show rank in results: Show rank number in results

```
WITH ranked AS (
  SELECT *, row_number() OVER (PARTITION BY symbol ORDER BY timestamp DESC) as rn
  FROM trades
  WHERE timestamp in today()
)
SELECT timestamp, symbol, price, rn as rank
FROM ranked
WHERE rn <= 5;
```

Run on demo.questdb.com

Alternative: Use negative LIMIT For a simpler approach when you need the latest N rows **total** (not per partition), use negative LIMIT:

Latest 100 trades overall (all symbols)

```
SELECT * FROM trades
WHERE symbol = 'BTC-USDT'
ORDER BY timestamp DESC
LIMIT 100;
```

Run on demo.questdb.com

Or more convenient with QuestDB's negative LIMIT feature:

Latest 100 trades using negative LIMIT

```
SELECT * FROM trades
WHERE symbol = 'BTC-USDT'
LIMIT -100;
```

Run on demo.questdb.com

But this doesn't work per partition - it returns 100 total rows, not 100 per symbol.

Performance optimization **Filter by timestamp first:**

```
-- Good: Reduces dataset before windowing
WITH ranked AS (
  SELECT *, row_number() OVER (PARTITION BY symbol ORDER BY timestamp DESC) as rn
  FROM trades
  WHERE timestamp in today()  -- Filter early
)
SELECT * FROM ranked WHERE rn <= 5;

-- Less efficient: Windows over entire table
WITH ranked AS (
  SELECT *, row_number() OVER (PARTITION BY symbol ORDER BY timestamp DESC) as rn
  FROM trades  -- No filter
)
SELECT * FROM ranked WHERE rn <= 5 AND timestamp in today();
```

Limit partitions:

```
-- Process only specific symbols
WHERE timestamp in today()
  AND symbol IN ('BTC-USDT', 'ETH-USDT', 'SOL-USDT')
```

Top N with aggregates Combine with aggregates to get summary statistics for top N:

Average price of latest 10 trades per symbol

```
WITH ranked AS (
  SELECT
    timestamp,
    symbol,
    price,
    row_number() OVER (PARTITION BY symbol ORDER BY timestamp DESC) as rn
  FROM trades
  WHERE timestamp in today()
)
SELECT
  symbol,
  count(*) as trade_count,
  avg(price) as avg_price,
  min(price) as min_price,
  max(price) as max_price
FROM ranked
WHERE rn <= 10
GROUP BY symbol;
```

Run on demo.questdb.com

Comparison with LATEST ON

Feature	LATEST ON	row_number() + Filter
Rows per partition	Exactly 1	Any number (N)
Performance	Very fast (optimized)	Moderate (requires ranking)
Flexibility	Fast	High (custom ordering, filtering)
Use case	Single latest value	Multiple recent values

View this recipe online

Calculate sessions and elapsed time

Calculate sessions and elapsed time by identifying when state changes occur in time-series data. This "flip-flop" or "session" pattern is useful for analyzing user sessions, vehicle rides, machine operating cycles, or any scenario where you need to track duration between state transitions.

Problem: Track time between state changes You have a table tracking vehicle lock status over time and want to calculate ride duration. A ride starts when `lock_status` changes from `true` (locked) to `false` (unlocked), and ends when it changes back to `true`.

Table schema:

```
CREATE TABLE vehicle_events (
  vehicle_id SYMBOL,
  lock_status BOOLEAN,
  timestamp TIMESTAMP
) TIMESTAMP(timestamp) PARTITION BY DAY;
```

Sample data:

timestamp	vehicle_id	lock_status
10:00:00	V001	true
10:05:00	V001	false
10:25:00	V001	true
10:30:00	V001	false
10:45:00	V001	true

You want to calculate the duration of each ride.

Solution: Session detection with window functions Use window functions to detect state changes, assign session IDs, then calculate durations:

```
WITH prevEvents AS (
  SELECT *,
    lag(lock_status::int) -- lag doesn't support booleans, so we convert to 1 or 0
      OVER (PARTITION BY vehicle_id ORDER BY timestamp) as prev_status
  FROM vehicle_events
  WHERE timestamp IN today()
),
ride_sessions AS (
  SELECT *,
    SUM(CASE
      WHEN lock_status = true AND prev_status = 0 THEN 1
      WHEN lock_status = false AND prev_status = 1 THEN 1
      ELSE 0
    END) OVER (PARTITION BY vehicle_id ORDER BY timestamp) as ride
  FROM prevEvents
),
global_sessions AS (
  SELECT *, concat(vehicle_id, '#', ride) as session
  FROM ride_sessions
),
totals AS (
  SELECT
```

```
    first(timestamp) as ts,
    session,
    FIRST(lock_status) as lock_status,
    first(vehicle_id) as vehicle_id
  FROM global_sessions
  GROUP BY session
),
prev_ts AS (
  SELECT *,
    lag(timestamp) OVER (PARTITION BY vehicle_id ORDER BY timestamp) as prev_ts
  FROM totals
)
SELECT
  timestamp as ride_end,
  vehicle_id,
  datediff('s', prev_ts, timestamp) as duration_seconds
FROM prev_ts
WHERE lock_status = false AND prev_ts IS NOT NULL;
```

Results:

ride_end	vehicle_id	duration_seconds
10:25:00	V001	1200
10:45:00	V001	900

How it works The query uses a five-step approach:

1. **Get previous status (`prevEvents`)**

```
lag(lock_status::int) OVER (PARTITION BY vehicle_id ORDER BY timestamp)
```

For each row, get the status from the previous row. Convert boolean to integer (0/1) since `lag` doesn't support boolean types directly.

2. **Detect state changes (`ride_sessions`)**

```
SUM(CASE WHEN lock_status != prev_status THEN 1 ELSE 0 END)
  OVER (PARTITION BY vehicle_id ORDER BY timestamp)
```

Whenever status changes, increment a counter. This creates sequential session IDs for each vehicle: - Ride 0: Initial state - Ride 1: After first state change - Ride 2: After second state change - …

3. **Create global session IDs (`global_sessions`)**

```
concat(vehicle_id, '#', ride)
```

Combine vehicle_id with ride number to create unique session identifiers across all vehicles.

4. **Get session start times (`totals`)**

```
SELECT first(timestamp) as ts, ...
FROM global_sessions
GROUP BY session
```

For each session, get the timestamp and status at the beginning of that session.

5. Calculate duration (`prev_ts`)

```
lag(timestamp) OVER (PARTITION BY vehicle_id ORDER BY timestamp)
```

Get the timestamp from the previous session (for the same vehicle), then use `datediff('s', prev_ts, timestamp)` to calculate duration in seconds.

Filter for rides

```
WHERE lock_status = false
```

Only show sessions where status is `false` (unlocked), which represents completed rides. The duration is from the previous session end (lock) to this session start (unlock).

Monthly aggregation Calculate total ride duration per vehicle per month:

```
WITH prevEvents AS (
  SELECT *,
    lag(lock_status::int) -- lag doesn't support booleans, so we convert to 1 or 0
      OVER (PARTITION BY vehicle_id ORDER BY timestamp) as prev_status
  FROM vehicle_events
  WHERE timestamp >= dateadd('M', -3, now())
),
ride_sessions AS (
  SELECT *,
    SUM(CASE
      WHEN lock_status = true AND prev_status = 0 THEN 1
      WHEN lock_status = false AND prev_status = 1 THEN 1
      ELSE 0
    END) OVER (PARTITION BY vehicle_id ORDER BY timestamp) as ride
  FROM prevEvents
),
global_sessions AS (
  SELECT *, concat(vehicle_id, '#', ride) as session
  FROM ride_sessions
),
totals AS (
  SELECT
    first(timestamp) as ts,
    session,
    FIRST(lock_status) as lock_status,
    first(vehicle_id) as vehicle_id
  FROM global_sessions
  GROUP BY session
),
prev_ts AS (
  SELECT *,
    lag(timestamp) OVER (PARTITION BY vehicle_id ORDER BY timestamp) as prev_ts
  FROM totals
)
SELECT
  timestamp_floor('M', timestamp) as month,
  vehicle_id,
  SUM(datediff('s', prev_ts, timestamp)) as total_ride_duration_seconds,
  COUNT(*) as ride_count
FROM prev_ts
WHERE lock_status = false AND prev_ts IS NOT NULL
GROUP BY month, vehicle_id
ORDER BY month, vehicle_id;
```

Adapting to different use cases User website sessions (1 hour timeout):

```
WITH prevEvents AS (
  SELECT *,
    lag(timestamp) OVER (PARTITION BY user_id ORDER BY timestamp) as prev_ts
  FROM page_views
),
sessions AS (
  SELECT *,
    SUM(CASE
      WHEN datediff('h', prev_ts, timestamp) > 1 THEN 1
      ELSE 0
    END) OVER (PARTITION BY user_id ORDER BY timestamp) as session_id
  FROM prevEvents
)
SELECT
  user_id,
  session_id,
  min(timestamp) as session_start,
  max(timestamp) as session_end,
  datediff('s', min(timestamp), max(timestamp)) as session_duration_seconds,
  count(*) as page_views
FROM sessions
GROUP BY user_id, session_id;
```

Machine operating cycles:

```
-- When machine changes from 'off' to 'running' to 'off'
WITH prevStatus AS (
  SELECT *,
    lag(status) OVER (PARTITION BY machine_id ORDER BY timestamp) as prev_status
  FROM machine_status
),
cycles AS (
  SELECT *,
    SUM(CASE
      WHEN status != prev_status THEN 1
      ELSE 0
    END) OVER (PARTITION BY machine_id ORDER BY timestamp) as cycle
  FROM prevStatus
)
SELECT
  machine_id,
  cycle,
  min(timestamp) as cycle_start,
  max(timestamp) as cycle_end
FROM cycles
WHERE status = 'running'
GROUP BY machine_id, cycle;
```

TIP: Common Session Patterns

This pattern applies to many scenarios: - **User sessions**: Time between last action and timeout - **IoT device cycles**: Power on/off cycles - **Vehicle trips**: Ignition on/off periods - **Connection sessions**: Login/logout tracking - **Process steps**: Start/complete state transitions

WARNING: First Row Handling

The first row in each partition will have `NULL` for previous values. Always filter these out with `WHERE prev_ts IS NOT NULL` or similar conditions.

View this recipe online

Query last N minutes of activity

Query data from the last N minutes of recorded activity in a table, regardless of the current time.

Problem You want to get data from a table for the last 15 minutes of activity.

You know you could do:

```
SELECT * FROM my_tb
WHERE timestamp > dateadd('m', -15, now());
```

But that would give you the last 15 minutes, not the last 15 minutes of activity in your table. Assuming the last timestamp recorded in your table was `2025-03-23T07:24:37.000000Z`, then you would like to get the data from `2025-03-23T07:09:37.000000Z` to `2025-03-23T07:24:37.000000Z`.

Solution Use a correlated subquery to find the latest timestamp, then filter relative to it:

```
SELECT *
FROM my_table
WHERE timestamp >= (
  SELECT dateadd('m', -15, timestamp)
  FROM my_table
  LIMIT -1
);
```

QuestDB supports correlated subqueries when asking for a timestamp if the query returns a scalar value. Using `LIMIT -1` we get the latest row in the table (sorted by designated timestamp), and we apply the `dateadd` function on that date, so it needs to be executed just once. If we placed the `dateadd` on the left, the calculation would need to be applied once for each row on the main table. This query should return in just a few milliseconds, independently of table size.

View this recipe online

Filter data by week number

Filter time-series data by week number using either the built-in `week_of_year()` function or `dateadd()` for better performance on large tables.

Solution 1: Using week_of_year() There is a built-in `week_of_year()` function, so this could be solved as:

Filter by week using week_of_year()

```
SELECT * FROM trades
WHERE week_of_year(timestamp) = 24;
```

Run on demo.questdb.com

Solution 2: Using dateadd() (faster) However, depending on your table size, especially if you are not filtering by any timestamp, you might prefer this alternative, as it executes faster:

Filter by week using dateadd()

```
SELECT * FROM trades
WHERE timestamp >= dateadd('w', 23, '2025-01-01')
  AND timestamp < dateadd('w', 24, '2025-01-01');
```

Run on demo.questdb.com

You need to be careful with that query, as it will start counting time from Jan 1st 1970, which is not a Monday.

Solution 3: Start at first Monday of year This alternative would start at the Monday of the week that includes January 1st:

Filter by week using first Monday calculation

```
DECLARE
  @year := '2025',
  @week := 24,
  @first_monday := dateadd('d', -1 * day_of_week(@year) + 1, @year),
  @week_start := dateadd('w', @week - 1, @first_monday),
  @week_end := dateadd('w', @week, @first_monday)
SELECT * FROM trades
WHERE timestamp >= @week_start
  AND timestamp < @week_end;
```

Run on demo.questdb.com
View this recipe online

Distribute discrete values across time intervals

Distribute discrete cumulative measurements across the time intervals between observations. When devices report cumulative values at irregular timestamps, you can spread those values proportionally across the intervals to get per-period averages.

This pattern is useful for scenarios like energy consumption, data transfer volumes, accumulated costs, or any metric where a cumulative value needs to be attributed to the intervals that contributed to it.

Problem You have IoT devices reporting watt-hour (Wh) values at irregular timestamps, identified by an `operationId`. You want to plot the sum of average power per operation, broken down by hour.

When an IoT device sends a `wh` value at discrete timestamps, you need to distribute that energy across the hours between measurements to visualize average power consumption per hour.

Raw data:

timestamp	operationId	wh
2025-04-01T14:10:59.000000Z	1001	0
2025-04-01T14:20:01.000000Z	1002	0
2025-04-01T15:06:29.000000Z	1003	0
2025-04-01T18:18:05.000000Z	1001	200
2025-04-01T20:06:36.000000Z	1003	200
2025-04-01T22:20:10.000000Z	1002	300

For operation 1001: 200 Wh consumed between 14:10:59 and 18:18:05 should be distributed across hours 14:00, 15:00, 16:00, 17:00, 18:00.

Solution Distribute watt-hours across hourly intervals

```
WITH
sampled AS (
  SELECT timestamp, operationId, sum(wh) as wh
  FROM meter
  SAMPLE BY 1h
  FILL(0)
),
sessions AS (
  SELECT *,
    SUM(CASE WHEN wh > 0 THEN 1 END)
      OVER (PARTITION BY operationId ORDER BY timestamp DESC) as session
  FROM sampled
),
counts AS (
  SELECT timestamp, operationId,
    FIRST_VALUE(wh) OVER (PARTITION BY operationId, session ORDER BY timestamp DESC) as wh,
    COUNT(*) OVER (PARTITION BY operationId, session) as attributable_hours
  FROM sessions
)
SELECT
  timestamp,
  operationId,
  wh / attributable_hours as wh_avg
FROM counts;
```

Run on demo.questdb.com

How it works:

The `sampled` subquery creates an entry for each operationId and missing hourly interval, filling with 0 wh for interpolated rows.

The key trick is dividing the data into "sessions". A session is defined by all the rows with no value for wh before a row with a value for wh. Or, if we reverse the timestamp order, a session would be defined by a row with a value for wh, followed by several rows with zero value for the same operationId:

```
SUM(CASE WHEN wh > 0 THEN 1 END) OVER (PARTITION BY operationId ORDER BY timestamp DESC) as session
```

For each operationId we get multiple sessions (1, 2, 3…). If we did:

```
COUNT() as attributable_hours GROUP BY operationId, session
```

We would get how many attributable rows each session has.

The `counts` subquery uses a window function to `COUNT` the number of rows per session (notice the count window function is not using `order by` so this will not be a running count, but all rows for the same session will have the same value as `attributable_hours`).

It also gets `FIRST_VALUE` for the session sorted by reverse timestamp, which is the `wh` value for the only row with value in each session.

The final query divides the `wh` reported in the session by the number of `attributable_hours`.

> **INFO: Filtering Results**
>
> If you want to filter the results by timestamp or operationId, you should add the filter at the first query (the one named `sampled`), so the rest of the process is done on the relevant subset of data.

View this recipe online

Query with epoch timestamps

Query using epoch timestamps instead of timestamp literals.

Problem You want to query data using epoch values rather than timestamp literals.

Solution Use epoch values directly in your WHERE clause. QuestDB expects microseconds by default for `timestamp` columns:

Query with epoch in microseconds

```
SELECT *
FROM trades
WHERE timestamp BETWEEN 1746552420000000 AND 1746811620000000;
```

Run on demo.questdb.com

> **INFO: Millisecond Resolution**
>
> If you have epoch values in milliseconds, you need to multiply by 1000 to convert to microseconds.

Nanoseconds can be used when the timestamp column is of type `timestamp_ns`.

Query with epoch in nanoseconds

```
SELECT *
FROM fx_trades
WHERE timestamp BETWEEN 1768303754000000000 AND 1778303754000000000;
```

Run on demo.questdb.com

> **NOTE: If the query does not return any data**
>
> Since the `fx_trades` table has a TTL, the query above may return empty results. To find valid epoch values with data, run:
>
> `select timestamp::long as from_epoch, dateadd('s', -10, timestamp)::long as to_epoch from fx_trades limit -1;`
>
> Then replace the `BETWEEN` values with the epochs returned.

View this recipe online

Right interval bound with SAMPLE BY

Use the right interval bound (end of interval) instead of the left bound (start of interval) for SAMPLE BY timestamps.

Problem Records are grouped in a 15-minute interval. For example, records between 2025-03-22T00:00:00.000000Z and 2025-03-22T00:15:00.000000Z are aggregated with timestamp 2025-03-22T00:00:00.000000Z.

You want the aggregation to show 2025-03-22T00:15:00.000000Z (the right bound of the interval rather than left).

Solution Simply shift the timestamp in the SELECT:

SAMPLE BY with right bound

```
SELECT
    dateadd('m', 15, timestamp) AS timestamp, symbol,
    first(price) AS open,
    last(price) AS close,
    min(price),
    max(price),
    sum(quantity) AS volume
FROM fx_trades
WHERE symbol = 'EURUSD' AND timestamp IN today()
SAMPLE BY 15m;
```

Run on demo.questdb.com
View this recipe online

Remove outliers from candle data

Remove outlier trades that differ significantly from recent average prices.

Problem You have candle data from trading pairs where some markets have very low volume trades that move the candle significantly. These are usually single trades with very low volume where the exchange rate differs a lot from the actual exchange rate. This makes charts hard to use and you would like to remove those from the chart.

Current query:

Daily OHLC candles

```
SELECT
    timestamp, symbol,
    first(price) AS open,
    last(price) AS close,
    min(price),
    max(price),
    sum(quantity) AS volume
FROM fx_trades
WHERE symbol = 'EURUSD' AND timestamp > dateadd('d', -14, now())
SAMPLE BY 1d;
```

Run on demo.questdb.com

The question is: is there a way to only select trades where the price deviates significantly from recent patterns?

Solution Use a window function to calculate a moving average of price, then filter out trades where the price deviates more than a threshold (e.g., 1%) from the moving average before aggregating with `SAMPLE BY`.

This query excludes trades where price deviates more than 1% from the 7-day moving average:

Filter outliers using 7-day moving average

```
WITH moving_trades AS (
  SELECT timestamp, symbol, price, quantity,
    avg(price) OVER (
      PARTITION BY symbol
      ORDER BY timestamp
      RANGE BETWEEN 7 days PRECEDING AND 1 day PRECEDING
    ) AS moving_avg_price
  FROM fx_trades
  WHERE symbol = 'EURUSD' AND timestamp > dateadd('d', -21, now())
)
SELECT
    timestamp, symbol,
    first(price) AS open,
    last(price) AS close,
    min(price),
    max(price),
    sum(quantity) AS volume
FROM moving_trades
WHERE timestamp > dateadd('d', -14, now())
  AND moving_avg_price IS NOT NULL
  AND ABS(price - moving_avg_price) <= moving_avg_price * 0.01
SAMPLE BY 1d;
```

Run on demo.questdb.com

NOTE: Moving Average Window

The CTE fetches 21 days of data (7 extra days) so the 7-day moving average window has enough history for the first rows in the 14-day result period.

View this recipe online

Fill missing intervals with value from another column

Fill missing intervals using the previous value from a specific column to populate multiple columns.

Problem You have a query like this:

SAMPLE BY with FILL(PREV)

```
SELECT timestamp, symbol, avg(bid_price) as bid_price, avg(ask_price) as ask_price
FROM core_price
WHERE symbol = 'EURUSD' AND timestamp IN today()
SAMPLE BY 100T FILL(PREV, PREV);
```

Run on demo.questdb.com

But when there is an interpolation, instead of getting the PREV value for `bid_price` and previous for `ask_price`, you want both prices to show the PREV known value for the `ask_price`. Imagine this SQL was valid:

```
SELECT timestamp, symbol, avg(bid_price) as bid_price, avg(ask_price) as ask_price
FROM core_price
WHERE symbol = 'EURUSD' AND timestamp IN today()
SAMPLE BY 100T FILL(PREV(ask_price), PREV);
```

Solution The only way to do this is in multiple steps within a single query: first get the sampled data interpolating with null values, then use a window function to get the last non-null value for the reference column, and finally coalesce the missing columns with this filler value.

Fill bid and ask prices with value from ask price

```
WITH sampled AS (
  SELECT timestamp, symbol, avg(bid_price) as bid_price, avg(ask_price) as ask_price
  FROM core_price
  WHERE symbol = 'EURUSD' AND timestamp IN today()
  SAMPLE BY 100T FILL(null)
), with_previous_vals AS (
  SELECT *,
    last_value(ask_price) IGNORE NULLS OVER(PARTITION BY symbol ORDER BY timestamp) as filler
  FROM sampled
)
SELECT timestamp, symbol, coalesce(bid_price, filler) as bid_price,
       coalesce(ask_price, filler) as ask_price
FROM with_previous_vals;
```

Run on demo.questdb.com

Note the use of `IGNORE NULLS` modifier on the window function to make sure we always look back for a value, rather than just over the previous row.

You can mark which rows were filled by adding a column that flags interpolated values:

Show which rows were filled

```
WITH sampled AS (
  SELECT timestamp, symbol, avg(bid_price) as bid_price, avg(ask_price) as ask_price
  FROM core_price
  WHERE symbol = 'EURUSD' AND timestamp IN today()
  SAMPLE BY 100T FILL(null)
), with_previous_vals AS (
  SELECT *,
    last_value(ask_price) IGNORE NULLS OVER(PARTITION BY symbol ORDER BY timestamp) as filler
```

```
  FROM sampled
)
SELECT timestamp, symbol, coalesce(bid_price, filler) as bid_price,
       coalesce(ask_price, filler) as ask_price,
       case when bid_price is NULL then true END as filled
FROM with_previous_vals;
```

Run on demo.questdb.com
View this recipe online

FILL PREV with historical values

When using `FILL(PREV)` with `SAMPLE BY` on a filtered time interval, gaps at the beginning may have null values because `PREV` only uses values from within the filtered interval. This recipe shows how to carry forward the last known value from before the interval.

Problem When you filter a time range and use `FILL(PREV)` or `FILL(LINEAR)`, QuestDB only considers values within the filtered interval. If the first sample bucket has no data, it will be null instead of carrying forward the last known value from before the interval.

Solution Use a "filler row" by querying the latest value before the filtered interval with `LIMIT -1`, then combine it with your filtered data using `UNION ALL`. The filler row provides the initial value for `FILL(PREV)` to use:

FILL with PREV values carried over last row before the time range in the WHERE

```
DECLARE
  @start_ts := dateadd('s', -3, now()),
  @end_ts := now()
WITH
filler_row AS (
  SELECT timestamp, open_mid AS open, high_mid AS high, close_mid AS close, low_mid AS low
  FROM core_price_1s
  WHERE timestamp < @start_ts
  LIMIT -1
),
sandwich AS (
  SELECT * FROM (
    SELECT * FROM filler_row
    UNION ALL
    SELECT timestamp, open_mid AS open, high_mid AS high, close_mid AS close, low_mid AS low
    FROM core_price_1s
    WHERE timestamp BETWEEN @start_ts AND @end_ts
  ) ORDER BY timestamp
),
sampled AS (
  SELECT
    timestamp,
    first(open) AS open,
    first(high) AS high,
    first(low) AS low,
    first(close) AS close
  FROM sandwich
  SAMPLE BY 100T
  FILL(PREV, PREV, PREV, PREV, 0)
)
SELECT * FROM sampled WHERE timestamp >= @start_ts;
```

Run on demo.questdb.com

This query: 1. Gets the latest row before the filtered interval using `LIMIT -1` (last row) 2. Combines it with filtered interval data using `UNION ALL` 3. Applies `SAMPLE BY` with `FILL(PREV)` - the filler row provides initial values 4. Filters results to exclude the filler row, keeping only the requested interval

The filler row ensures that gaps at the beginning of the interval carry forward the last known value rather than showing nulls.

View this recipe online

FILL on keyed queries with arbitrary intervals

You want to sample data and fill any potential gaps with interpolated values, using a time interval defined by a starting and ending timestamp, not only between the first and last existing row in the filtered results.

Problem QuestDB has a built-in `SAMPLE BY .. FROM/TO` syntax available for non-keyed queries (queries that include only aggregated columns beyond the timestamp), and for the `NULL` FILL strategy.

If you use `FROM/TO` in a keyed query (for example, an OHLC with timestamp, symbol, and aggregations) you will get the following error: *FROM-TO intervals are not supported for keyed SAMPLE BY queries*.

Solution "Sandwich" your data by adding artificial boundary rows at the start and end of your time interval using `UNION ALL`. These rows contain your target timestamps with nulls for all other columns. Then you can use `FILL` without the `FROM/TO` keywords and get results for every sampled interval within those arbitrary dates.

FILL arbitrary interval with keyed SAMPLE BY

```
DECLARE
  @start_ts := dateadd('m', -2, now()),
  @end_ts := dateadd('m', 2, now())
WITH
sandwich AS (
  SELECT * FROM (
    SELECT @start_ts AS timestamp, null AS symbol, null AS open, null AS high, null AS close, null AS l
    UNION ALL
    SELECT timestamp, symbol, open_mid AS open, high_mid AS high, close_mid AS close, low_mid AS low
    FROM core_price_1s
    WHERE timestamp BETWEEN @start_ts AND @end_ts
    UNION ALL
    SELECT @end_ts AS timestamp, null AS symbol, null AS open, null AS high, null AS close, null AS low
  ) ORDER BY timestamp
),
sampled AS (
  SELECT
    timestamp,
    symbol,
    first(open) AS open,
    first(high) AS high,
    first(low) AS low,
    first(close) AS close
  FROM sandwich
  SAMPLE BY 30s
  FILL(PREV, PREV, PREV, PREV, 0)
)
SELECT * FROM sampled WHERE open IS NOT NULL AND symbol IN ('EURUSD', 'GBPUSD');
```

Run on demo.questdb.com

This query: 1. Creates boundary rows with null values at the start and end timestamps 2. Combines them with filtered data using `UNION ALL` 3. Applies `ORDER BY timestamp` to preserve the designated timestamp 4. Performs `SAMPLE BY` with `FILL` - gaps are filled across the full interval 5. Filters out the artificial boundary rows using `open IS NOT NULL`

The boundary rows ensure that gaps are filled from the beginning to the end of your specified interval, not just between existing data points.

View this recipe online

Join strategies for sparse sensor data

Efficiently query sparse sensor data by splitting wide tables into narrow tables and joining them with different strategies.

Problem You have a sparse sensors table with 120 sensor columns, in which you are getting just a few sensor values at any given timestamp, so most values are null.

When you want to query data from any given sensor, you first SAMPLE the data with an `avg` or a `last_not_null` function aggregation, and then often build a CTE and call `LATEST ON` to get results:

```
SELECT
  timestamp,
  vehicle_id,
  avg(sensor_1) AS avg_sensor_1, avg(sensor_2) AS avg_sensor_2,
  ...
  avg(sensor_119) AS avg_sensor_119, avg(sensor_120) AS avg_sensor_120
FROM
  vehicle_sensor_data
-- WHERE vehicle_id = 'AAA0000'
SAMPLE BY 30s
LIMIT 100000;
```

This works, but it is not super fast (1sec for 10 million rows, in a table with 120 sensor columns and with 10k different vehicle_ids), and it is also not very efficient because `null` columns take some bytes on disk.

Solution: Multiple narrow tables with joins A single table works, but there is a more efficient (although a bit more cumbersome if you compose queries by hand) way to do this.

You can create 120 tables, one per sensor, rather than a table with 120 columns. Well, technically you probably want 121 tables, one with the common dimensions, then 1 per sensor. Or maybe you want N tables, one for the common dimensions, then N depending on how many sensor groups you have, as some groups might always send in sync. In any case, rather than a wide table you would end up with several narrow tables that you would need to join.

Now for joining the tables there are three potential ways, depending on the results you are after: * To see the *LATEST* known value for all the metrics *for a given series*, use a `CROSS JOIN` strategy (example below). This returns a single row. * To see the *LATEST* known value for all the metrics and *for all or several series*, use a `LEFT JOIN` strategy. This returns a single row per series (example below). * To see the *rolling view of all the latest known values* regarding the current row for one of the metrics, use an `ASOF JOIN` strategy. This returns as many rows as you have in the main metric you are querying (example below).

Performance The three approaches perform well. The three queries were executed on a table like the initial one, with 10 million rows representing sparse data from 10k series and across 120 metrics, so 120 tables. Each of the 120 tables had ~83k records (which times 120 is ~10 million rows).

`CROSS JOIN` is the fastest, executing in 23ms, `ASOF JOIN` is second with 123 ms, and `LEFT JOIN` is the slowest at 880ms. Still not too bad, as you probably will not want to get all the sensors from all the devices all the time, and joining fewer tables would perform better.

Strategy 1: CROSS JOIN We first find the latest point in each of the 120 tables for the given series (AAA0000), so we get a value per table, and then do a `CROSS JOIN`, to get a single row.

```
WITH
s1 AS (SELECT timestamp, vehicle_id, value FROM vehicle_sensor_1
    WHERE vehicle_id = 'AAA0000' LATEST ON timestamp PARTITION BY vehicle_id),
s2 AS (SELECT timestamp, vehicle_id, value FROM vehicle_sensor_2
    WHERE vehicle_id = 'AAA0000' LATEST ON timestamp PARTITION BY vehicle_id),
...
s119 AS (SELECT timestamp, vehicle_id, value FROM vehicle_sensor_119
    WHERE vehicle_id = 'AAA0000' LATEST ON timestamp PARTITION BY vehicle_id),
s120 AS (SELECT timestamp, vehicle_id, value FROM vehicle_sensor_120
    WHERE vehicle_id = 'AAA0000' LATEST ON timestamp PARTITION BY vehicle_id)
SELECT s1.timestamp, s1.vehicle_id, s1.value AS value_1,
s2.value AS value_2,
...
s119.value AS value_119,
s120.value AS value_120
FROM s1
CROSS JOIN s2
CROSS JOIN ...
CROSS JOIN s119
CROSS JOIN s120;
```

Strategy 2: LEFT JOIN We first find the latest point in each of the 120 tables for each series, so we get a value per table and series, and then do a `LEFT JOIN` on the series ID, to get a single row for each different series (10K rows in our example).

```
WITH
s1 AS (SELECT timestamp, vehicle_id, value FROM vehicle_sensor_1
    LATEST ON timestamp PARTITION BY vehicle_id),
s2 AS (SELECT timestamp, vehicle_id, value FROM vehicle_sensor_2
    LATEST ON timestamp PARTITION BY vehicle_id),
...
s119 AS (SELECT timestamp, vehicle_id, value FROM vehicle_sensor_119
    LATEST ON timestamp PARTITION BY vehicle_id),
s120 AS (SELECT timestamp, vehicle_id, value FROM vehicle_sensor_120
    LATEST ON timestamp PARTITION BY vehicle_id)
SELECT s1.timestamp, s1.vehicle_id, s1.value AS value_1,
s2.value AS value_2,
...
s119.value AS value_119,
s120.value AS value_120
FROM s1
LEFT JOIN s2 ON s1.vehicle_id = s2.vehicle_id
LEFT JOIN ...
LEFT JOIN s119 ON s1.vehicle_id = s119.vehicle_id
LEFT JOIN s120 ON s1.vehicle_id = s120.vehicle_id;
```

Strategy 3: ASOF JOIN We get all the rows in all the tables, then do an `ASOF JOIN` on the series ID, so we get a row for each row of the first table in the query, in our example ~83K results.

```
WITH
s1 AS (SELECT timestamp, vehicle_id, value FROM vehicle_sensor_1 ),
s2 AS (SELECT timestamp, vehicle_id, value FROM vehicle_sensor_2 ),
...
s118 AS (SELECT timestamp, vehicle_id, value FROM vehicle_sensor_118 ),
s119 AS (SELECT timestamp, vehicle_id, value FROM vehicle_sensor_119 ),
s120 AS (SELECT timestamp, vehicle_id, value FROM vehicle_sensor_120 )
```

```
SELECT s1.timestamp, s1.vehicle_id, s1.value AS value_1,
       s2.value AS value_2,
       ...
       s119.value AS value_119,
       s120.value AS value_120
FROM s1
ASOF JOIN s2 ON s1.vehicle_id = s2.vehicle_id
ASOF JOIN ...
ASOF JOIN s119 ON s1.vehicle_id = s119.vehicle_id
ASOF JOIN s120 ON s1.vehicle_id = s120.vehicle_id;
```

View this recipe online

Advanced Queries

Access rows before and after current row

Access values from rows before and after the current row to find patterns, detect changes, or provide context around events.

Problem You want to see values from surrounding rows alongside the current row - for example, the 5 previous and 5 next bid prices for each row.

Solution Use `LAG()` to access rows before the current row and `LEAD()` to access rows after:

Access surrounding row values with LAG and LEAD

```
SELECT timestamp, bid_price,
  LAG(bid_price, 1) OVER () AS prev_1,
  LAG(bid_price, 2) OVER () AS prev_2,
  LAG(bid_price, 3) OVER () AS prev_3,
  LAG(bid_price, 4) OVER () AS prev_4,
  LAG(bid_price, 5) OVER () AS prev_5,
  LEAD(bid_price, 1) OVER () AS next_1,
  LEAD(bid_price, 2) OVER () AS next_2,
  LEAD(bid_price, 3) OVER () AS next_3,
  LEAD(bid_price, 4) OVER () AS next_4,
  LEAD(bid_price, 5) OVER () AS next_5
FROM core_price
WHERE timestamp >= dateadd('m', -1, now()) AND symbol = 'EURUSD';
```

Run on demo.questdb.com

How it works

- **`LAG(column, N)`** - Gets the value from N rows **before** the current row (earlier in time)
- **`LEAD(column, N)`** - Gets the value from N rows **after** the current row (later in time)

Both functions return `NULL` for rows where the offset goes beyond the dataset boundaries (e.g., `LAG(5)` returns `NULL` for the first 5 rows).

View this recipe online

Find local minimum and maximum

Find the minimum and maximum values within a time window around each row to detect local peaks, troughs, or price ranges.

Problem You want to find the local minimum and maximum bid price within a time range of each row - for example, the min/max within 1 second before and after each data point.

Solution 1: Window function (past only) If you only need to look at **past data**, use a window function with `RANGE`:

Local min/max from preceding 1 second

```
SELECT timestamp, bid_price,
  min(bid_price) OVER (ORDER BY timestamp RANGE 1 second PRECEDING) AS min_price,
  max(bid_price) OVER (ORDER BY timestamp RANGE 1 second PRECEDING) AS max_price
FROM core_price
WHERE timestamp >= dateadd('m', -1, now()) AND symbol = 'EURUSD';
```

Run on demo.questdb.com

This returns the minimum and maximum bid price from the 1 second preceding each row.

Solution 2: WINDOW JOIN (past and future) If you need to look at **both past and future data**, use a `WINDOW JOIN`. QuestDB window functions don't support `FOLLOWING`, but WINDOW JOIN allows bidirectional lookback:

Local min/max from 1 second before and after

```
SELECT p.timestamp, p.bid_price,
  min(pp.bid_price) AS min_price,
  max(pp.bid_price) AS max_price
FROM core_price p
WINDOW JOIN core_price pp ON symbol
  RANGE BETWEEN 1 second PRECEDING AND 1 second FOLLOWING
WHERE p.timestamp >= dateadd('m', -1, now()) AND p.symbol = 'EURUSD';
```

Run on demo.questdb.com

This returns the minimum and maximum bid price from 1 second before to 1 second after each row.

When to use each approach

Approach	Use When
Window function	You only need to look at past data
WINDOW JOIN	You need to look at both past and future data

View this recipe online

Top N plus others row

Create aggregated results showing the top N items individually, with all remaining items combined into a single "Others" row. This pattern is useful for dashboards and reports where you want to highlight the most important items while still showing the total.

Problem: Show top items plus remainder You want to display results like:

symbol	total_trades
BTC-USDT	15234
ETH-USDT	12890
SOL-USDT	8945
MATIC-USDT	6723
AVAX-USDT	5891
-Others-	23456

Instead of listing all symbols (which might be thousands), show the top 5 individually and aggregate the rest.

Solution: Use rank() with CASE statement Use `rank()` to identify top N rows, then use `CASE` to group remaining rows:

Top 5 symbols plus Others

```
WITH totals AS (
  SELECT
    symbol,
    count() as total
  FROM trades
  WHERE timestamp >= dateadd('d', -1, now())
),
ranked AS (
  SELECT
    *,
    rank() OVER (ORDER BY total DESC) as ranking
  FROM totals
)
SELECT
  CASE
    WHEN ranking <= 5 THEN symbol
    ELSE '-Others-'
  END as symbol,
  SUM(total) as total_trades
FROM ranked
GROUP BY 1
ORDER BY total_trades DESC;
```

Run on demo.questdb.com

Results:

symbol	total_trades
BTC-USDT	15234
ETH-USDT	12890

symbol	total_trades
SOL-USDT	8945
MATIC-USDT	6723
AVAX-USDT	5891
-Others-	23456

How it works The query uses a three-step approach:

1. **Aggregate data** (`totals` CTE):
   - Count or sum values by the grouping column
   - Creates base data for ranking
2. **Rank rows** (`ranked` CTE):
   - `rank() OVER (ORDER BY total DESC)`: Assigns rank based on count (1 = highest)
   - Ties receive the same rank
3. **Conditional grouping** (outer query):
   - `CASE WHEN ranking <= 5`: Keep top 5 with original names
   - `ELSE '-Others-'`: Rename all others to "-Others-"
   - `SUM(total)`: Aggregate counts (combines all "Others" into one row)
   - `GROUP BY 1`: Group by the CASE expression result

Understanding rank() `rank()` assigns ranks with gaps for ties:

symbol	total	rank
BTC-USDT	1000	1
ETH-USDT	900	2
SOL-USDT	900	2
AVAX-USDT	800	4
MATIC-USDT	700	5

If there are ties at the boundary (rank 5), all tied items will be included in top N.

Alternative: Using row_number() If you don't want to handle ties and always want exactly N rows in top tier:

Top 5 symbols, discarding extra buckets in case of a match

```
WITH totals AS (
  SELECT symbol, count() as total
  FROM trades
),
ranked AS (
  SELECT *, row_number() OVER (ORDER BY total DESC) as rn
  FROM totals
)
SELECT
  CASE WHEN rn <= 5 THEN symbol ELSE '-Others-' END as symbol,
  SUM(total) as total_trades
FROM ranked
GROUP BY 1
ORDER BY total_trades DESC;
```

Run on demo.questdb.com

Difference: - `rank()`: May include more than N if there are ties at position N - `row_number()`: Always exactly N in top tier (breaks ties arbitrarily)

Adapting the pattern Different top N:

```
-- Top 10 instead of top 5
WHEN ranking <= 10 THEN symbol

-- Top 3
WHEN ranking <= 3 THEN symbol
```

Different aggregations:

```
-- Sum instead of count
WITH totals AS (
  SELECT symbol, SUM(amount) as total_volume
  FROM trades
)
...
```

Multiple levels:

```
SELECT
  CASE
    WHEN ranking <= 5 THEN symbol
    WHEN ranking <= 10 THEN '-Top 10-'
    ELSE '-Others-'
  END as category,
  SUM(total) as count
FROM ranked
GROUP BY 1;
```

Results in three groups: top 5 individual, ranks 6-10 combined, rest combined.

With percentage: Top 5 symbols with percentage of total

```
WITH totals AS (
  SELECT symbol, count() as total
  FROM trades
  WHERE timestamp >= dateadd('d', -1, now())
),
ranked AS (
  SELECT *, rank() OVER (ORDER BY total DESC) as ranking
  FROM totals
),
summed AS (
  SELECT SUM(total) as grand_total FROM totals
),
grouped AS (
  SELECT
    CASE WHEN ranking <= 5 THEN symbol ELSE '-Others-' END as symbol,
    SUM(total) as total_trades
  FROM ranked
  GROUP BY 1
)
SELECT
  symbol,
```

```
  total_trades,
  round(100.0 * total_trades / grand_total, 2) as percentage
FROM grouped CROSS JOIN summed
ORDER BY total_trades DESC;
```

Run on demo.questdb.com

Multiple grouping columns Show top N for multiple dimensions:

Top 3 for each symbol and side

```
WITH totals AS (
  SELECT
    symbol,
    side,
    count() as total
  FROM trades
  WHERE timestamp >= dateadd('d', -1, now())
),
ranked AS (
  SELECT
    *,
    rank() OVER (PARTITION BY side ORDER BY total DESC) as ranking
  FROM totals
)
SELECT
  side,
  CASE WHEN ranking <= 3 THEN symbol ELSE '-Others-' END as symbol,
  SUM(total) as total_trades
FROM ranked
GROUP BY side, 2
ORDER BY side, total_trades DESC;
```

Run on demo.questdb.com

This shows top 3 symbols separately for buy and sell sides.

Visualization considerations This pattern is particularly useful for charts:

Pie/Donut charts:

```
-- Top 5 slices plus "Others" slice
CASE WHEN ranking <= 5 THEN symbol ELSE '-Others-' END
```

Bar charts:

```
-- Top 10 bars, sorted by value
CASE WHEN ranking <= 10 THEN symbol ELSE '-Others-' END
ORDER BY total_trades DESC
```

WARNING: Empty Others Row

If there are N or fewer distinct values, the “Others” row won’t appear (or will have 0 count). Handle this in your visualization logic if needed.

View this recipe online

Pivot with "Others" column

QuestDB has a native PIVOT keyword for transforming rows into columns. However, when you need to pivot specific values while grouping all remaining values into an "Others" column, you need to use `CASE` statements instead.

Problem You want to pivot data so that specific symbols (like EURUSD, GBPUSD, USDJPY) become columns, but also capture all other symbols in a single "Others" column:

Aggregated data per symbol

```
SELECT timestamp, symbol, SUM(bid_volume) AS total_bid
FROM core_price
WHERE timestamp IN today()
SAMPLE BY 1m
LIMIT 20;
```

Run on demo.questdb.com

Results:

timestamp	symbol	total_bid
2026-01-11T00:00:00.000000Z	EURGBP	124820733
2026-01-11T00:00:00.000000Z	AUDUSD	124778371
2026-01-11T00:00:00.000000Z	GBPAUD	124645353
2026-01-11T00:00:00.000000Z	GBPNZD	129175334
2026-01-11T00:00:00.000000Z	NZDUSD	127053437
2026-01-11T00:00:00.000000Z	USDSGD	130915407
2026-01-11T00:00:00.000000Z	USDJPY	123039292
2026-01-11T00:00:00.000000Z	AUDCAD	121234190
2026-01-11T00:00:00.000000Z	USDMXN	122254886
2026-01-11T00:00:00.000000Z	USDSEK	129272298
2026-01-11T00:00:00.000000Z	USDNOK	124493591
2026-01-11T00:00:00.000000Z	EURJPY	126254805
2026-01-11T00:00:00.000000Z	CADJPY	133359111
2026-01-11T00:00:00.000000Z	EURCHF	125818826
2026-01-11T00:00:00.000000Z	GBPJPY	130940614
2026-01-11T00:00:00.000000Z	USDCAD	126619566
2026-01-11T00:00:00.000000Z	USDTRY	124860359
2026-01-11T00:00:00.000000Z	AUDJPY	135946504
2026-01-11T00:00:00.000000Z	NZDJPY	126419110
2026-01-11T00:00:00.000000Z	EURAUD	122966167

Solution Use `CASE` statements with `NOT IN` for the "Others" column:

Pivot with Others column

```
SELECT timestamp,
  SUM(CASE WHEN symbol = 'EURUSD' THEN bid_volume END) AS EURUSD,
  SUM(CASE WHEN symbol = 'GBPUSD' THEN bid_volume END) AS GBPUSD,
  SUM(CASE WHEN symbol = 'USDJPY' THEN bid_volume END) AS USDJPY,
  SUM(CASE WHEN symbol NOT IN ('EURUSD', 'GBPUSD', 'USDJPY')
    THEN bid_volume END) AS OTHERS
FROM core_price
WHERE timestamp IN today()
SAMPLE BY 1m
LIMIT 5;
```

Run on demo.questdb.com
Results:

timestamp	EURUSD	GBPUSD	USDJPY	OTHERS
2026-01-11T00:00:00.000000Z	123717175	123252388	123039292	2755547557
2026-01-11T00:01:00.000000Z	130947736	136509565	127006858	2877498962
2026-01-11T00:02:00.000000Z	130004490	125804660	122451477	2824893498
2026-01-11T00:03:00.000000Z	124303244	126589046	124435638	2797775211
2026-01-11T00:04:00.000000Z	120743669	127991352	122970185	2733242883

Each timestamp now has a single row with specific symbols as columns, plus an "Others" column aggregating all remaining symbols.

When to use this pattern Use `CASE` statements instead of `PIVOT` when you need: - An "Others" or "Else" column to catch unspecified values - Custom aggregation logic per column - Different aggregation functions for different columns

For straightforward pivoting without an "Others" column, use the native PIVOT keyword.

View this recipe online

Unpivoting query results

Transform wide-format data (multiple columns) into long format (rows) using UNION ALL.

Problem: Wide format to long format You have query results with multiple columns where only one column has a value per row:

Wide format (sparse):

timestamp	symbol	buy	sell
08:10:00	ETH-USDT	NULL	3678.25
08:10:00	ETH-USDT	NULL	3678.25
08:10:00	ETH-USDT	3678.01	NULL
08:10:00	ETH-USDT	NULL	3678.00

You want to convert this to a format where side and price are explicit:

Long format (dense):

timestamp	symbol	side	price
08:10:00	ETH-USDT	sell	3678.25
08:10:00	ETH-USDT	sell	3678.25
08:10:00	ETH-USDT	buy	3678.01
08:10:00	ETH-USDT	sell	3678.00

Solution: UNION ALL with literal values Use UNION ALL to stack columns as rows, then filter NULL values:

UNPIVOT buy/sell columns to side/price rows

```
WITH pivoted AS (
  SELECT
    timestamp,
    symbol,
    CASE WHEN side = 'buy' THEN price END as buy,
    CASE WHEN side = 'sell' THEN price END as sell
  FROM trades
  WHERE timestamp >= dateadd('m', -5, now())
    AND symbol = 'ETH-USDT'
),
unpivoted AS (
  SELECT timestamp, symbol, 'buy' as side, buy as price
  FROM pivoted

  UNION ALL

  SELECT timestamp, symbol, 'sell' as side, sell as price
  FROM pivoted
)
SELECT * FROM unpivoted
WHERE price IS NOT NULL
ORDER BY timestamp;
```

Run on demo.questdb.com

Results:

timestamp	symbol	side	price
08:10:00	ETH-USDT	sell	3678.25
08:10:00	ETH-USDT	sell	3678.25
08:10:00	ETH-USDT	buy	3678.01
08:10:00	ETH-USDT	sell	3678.00

How it works

Step 1: Create wide format (if needed) If your data is already in narrow format, you may need to pivot first:

```
CASE WHEN side = 'buy' THEN price END as buy,
CASE WHEN side = 'sell' THEN price END as sell
```

This creates NULL values for the opposite side.

Step 2: UNION ALL

```
SELECT timestamp, symbol, 'buy' as side, buy as price FROM pivoted
UNION ALL
SELECT timestamp, symbol, 'sell' as side, sell as price FROM pivoted
```

This creates two copies of every row: - First copy: Has 'buy' literal with buy column value - Second copy: Has 'sell' literal with sell column value

Step 3: Filter NULLs

```
WHERE price IS NOT NULL
```

Removes rows where the price column is NULL (the opposite side).

Unpivoting multiple columns Transform multiple numeric columns to name-value pairs:

```
WITH sensor_data AS (
  SELECT
    timestamp,
    sensor_id,
    temperature,
    humidity,
    pressure
  FROM sensors
  WHERE timestamp >= dateadd('h', -1, now())
)
SELECT timestamp, sensor_id, 'temperature' as metric, temperature as value FROM sensor_data
WHERE temperature IS NOT NULL

UNION ALL

SELECT timestamp, sensor_id, 'humidity' as metric, humidity as value FROM sensor_data
WHERE humidity IS NOT NULL

UNION ALL

SELECT timestamp, sensor_id, 'pressure' as metric, pressure as value FROM sensor_data
```

```
WHERE pressure IS NOT NULL

ORDER BY timestamp, sensor_id, metric;
```

Results:

timestamp	sensor_id	metric	value
10:00:00	S001	humidity	65.2
10:00:00	S001	pressure	1013.2
10:00:00	S001	temperature	22.5

Performance considerations UNION ALL vs UNION:

```
-- Fast: UNION ALL (no deduplication)
SELECT ... UNION ALL SELECT ...

-- Slower: UNION (deduplicates rows)
SELECT ... UNION SELECT ...
```

Always use `UNION ALL` for unpivoting unless you specifically need deduplication.
View this recipe online

Sankey and funnel diagrams

Build user journey flow data for Sankey diagrams and conversion funnels by sessionizing event data and tracking state transitions.

Problem You want to build a user-flow or Sankey diagram to find out which pages contribute visits to others, and in which proportion. You'd like to track elapsed time, number of pages in a single session, entry/exit pages, etc., similar to web analytics tools.

Your issue is that you only capture a flat table with events, with no concept of session. For analytics purposes, you want to define a session as a visit that was more than 1 hour apart from the last one for the same user.

Your simplified table schema:

```
CREATE TABLE events (
    visitor_id SYMBOL,
    pathname SYMBOL,
    timestamp TIMESTAMP,
    metric_name SYMBOL
) TIMESTAMP(timestamp) PARTITION BY MONTH WAL;
```

Solution: Session window functions By combining window functions and `CASE` statements:

1. Sessionize the data by identifying gaps longer than 1 hour
2. Generate unique session ids for aggregations
3. Assign sequence numbers to each hit within a session
4. Assign the session initial timestamp
5. Check next page in the sequence

With that, you can count page hits for the next page from current, identify elapsed time between hits or since the start of the session, count sessions per user, or power navigation funnels and Sankey diagrams.

```
WITH PrevEvents AS (
  SELECT
    visitor_id,
    pathname,
    timestamp,
    lag(timestamp) OVER (PARTITION BY visitor_id ORDER BY timestamp) AS prev_ts
  FROM
    events WHERE timestamp > dateadd('d', -7, now())
    AND metric_name = 'page_view'
), VisitorSessions AS (
  SELECT *,
    SUM(CASE WHEN datediff('h', timestamp, prev_ts) > 1 THEN 1 END)
    OVER(
      PARTITION BY visitor_id
      ORDER BY timestamp
    ) as local_session_id FROM PrevEvents

), GlobalSessions AS (
  SELECT visitor_id, pathname, timestamp, prev_ts,
    concat(visitor_id, '#', coalesce(local_session_id,0)::int) AS session_id
  FROM VisitorSessions
), EventSequences AS (
  SELECT *, row_number() OVER (
```

```
      PARTITION BY session_id ORDER BY timestamp
    ) as session_sequence,
    row_number() OVER (
      PARTITION BY session_id ORDER BY timestamp DESC
    ) as reverse_session_sequence,
    first_value(timestamp::long) OVER (
      PARTITION BY session_id ORDER BY timestamp
    ) as session_ts
  FROM GlobalSessions
), EventsFullInfo AS (
  SELECT e1.session_id, e1.session_ts::timestamp as session_ts, e1.visitor_id,
    e1.timestamp, e1.pathname, e1.session_sequence,
    CASE WHEN e1.session_sequence = 1 THEN true END is_entry_page,
    e2.pathname as next_pathname, datediff('T', e1.timestamp, e1.prev_ts)::double as elapsed,
    e2.reverse_session_sequence,
    CASE WHEN e2.reverse_session_sequence = 1 THEN true END is_exit_page
  FROM EventSequences e1
  LEFT JOIN EventSequences e2 ON (e1.session_id = e2.session_id)
  WHERE e2.session_sequence - e1.session_sequence = 1
)
SELECT * FROM EventsFullInfo;
```

View this recipe online

Multiple conditional aggregates

Calculate multiple aggregates with different conditions in a single pass through the data using CASE expressions.

Problem You need to calculate various metrics from the same dataset with different conditions: - Count of buy orders - Count of sell orders - Average buy price - Average sell price - Total volume for large trades (> 1.0) - Total volume for small trades (≤ 1.0)

Running separate queries is inefficient.

Solution: CASE within aggregate functions Use CASE expressions inside aggregates to calculate all metrics in one query:

Multiple conditional aggregates in single query

```
SELECT
  symbol,
  count(CASE WHEN side = 'buy' THEN 1 END) as buy_count,
  count(CASE WHEN side = 'sell' THEN 1 END) as sell_count,
  avg(CASE WHEN side = 'buy' THEN price END) as avg_buy_price,
  avg(CASE WHEN side = 'sell' THEN price END) as avg_sell_price,
  sum(CASE WHEN amount > 1.0 THEN amount END) as large_trade_volume,
  sum(CASE WHEN amount <= 1.0 THEN amount END) as small_trade_volume,
  sum(amount) as total_volume
FROM trades
WHERE timestamp >= dateadd('d', -1, now())
  AND symbol IN ('BTC-USDT', 'ETH-USDT')
GROUP BY symbol;
```

Run on demo.questdb.com

Which returns:

symbol	buy_count	sell_count	avg_buy_price	avg_sell_price	large_trade_volume	small_trade_volume	total_volume
ETH-USDT	262870	212163	3275.286678132736	3274.747631752642	1.0215079994	9.917160999976	10.93866900489
BTC-USDT	789959	712152	94286.52121799352	94282.92124327847	1241887299995	1.057609990726	1242.629949730019

How it works

CASE returns NULL for non-matching rows

```
count(CASE WHEN side = 'buy' THEN 1 END)
```

- When `side = 'buy'`: CASE returns 1
- When `side != 'buy'`: CASE returns NULL (implicit ELSE NULL)
- `count()` only counts non-NULL values
- Result: counts only rows where side is 'buy'

Aggregate functions ignore NULL

```
avg(CASE WHEN side = 'buy' THEN price END)
```

- `avg()` calculates average of non-NULL values only
- Only includes price when side is 'buy'
- Automatically skips all other rows

View this recipe online

General and sampled aggregates

Combine overall (unsampled) aggregates with sampled aggregates in the same query.

Problem You have a query with three aggregates:

Max and Min

```
SELECT max(price), avg(price), min(price)
FROM trades
WHERE timestamp IN '2024-12-08';
```

Run on demo.questdb.com

This returns:

```
| max(price) | avg(price)         | min(price)   |
| ---------- | ------------------ | ------------ |
| 101464.2   | 15816.513123255792 | 0.000031204 |
```

And another query to get event count per second, then select the maximum:

Sample by 1m and get the top result

```
SELECT max(count_sec) FROM (
  SELECT count() as count_sec FROM trades
  WHERE timestamp IN '2024-12-08'
  SAMPLE BY 1s
);
```

Run on demo.questdb.com

This returns:

```
| max(count_sec) |
| -------------- |
| 4473           |
```

You want to combine both results in a single row:

```
| max(count_sec) | max(price) | avg(price)         | min(price)   |
| -------------- | ---------- | ------------------ | ------------ |
| 4473           | 101464.2   | 15816.513123255792 | 0.000031204 |
```

Solution: CROSS JOIN A `CROSS JOIN` can join every row from the first query (1 row) with every row from the second (1 row), so you get a single row with all the aggregates combined:

Combine general and sampled aggregates

```
WITH
max_min AS (
SELECT max(price), avg(price), min(price)
FROM trades WHERE timestamp IN '2024-12-08'
)
SELECT max(count_sec), max_min.* FROM (
  SELECT count() as count_sec FROM trades
  WHERE timestamp IN '2024-12-08'
  SAMPLE BY 1s
) CROSS JOIN max_min;
```

Run on demo.questdb.com

View this recipe online

Consistent histogram buckets

Create histograms with consistent bucket boundaries for distribution analysis. Different approaches suit different data characteristics.

Problem A fixed bucket size works well for some data but poorly for others. For example, a bucket size of 0.5 produces a nice histogram for BTC trade amounts, but may produce just one or two buckets for assets with smaller typical values.

Solution 1: Fixed bucket size When you know your data range, use a fixed bucket size:

Histogram with fixed 0.5 buckets

```
DECLARE @bucket_size := 0.5
SELECT
  floor(amount / @bucket_size) * @bucket_size AS bucket,
  count(*) AS count
FROM trades
WHERE symbol = 'BTC-USDT' AND timestamp IN today()
GROUP BY bucket
ORDER BY bucket;
```

Run on demo.questdb.com

How it works

```
floor(amount / 0.5) * 0.5
```

1. `amount / 0.5`: Divide by bucket width (1.3 □ 2.6)
2. `floor(...)`: Truncate to integer (2.6 □ 2)
3. `* 0.5`: Multiply back (2 □ 1.0)

Examples: - 0.3 □ floor(0.6) × 0.5 = 0.0 - 1.3 □ floor(2.6) × 0.5 = 1.0 - 2.7 □ floor(5.4) × 0.5 = 2.5

NOTE: You must tune `@bucket_size` for your data range. A size that works for one symbol may not work for another.

Solution 2: Fixed bucket count (dynamic size) To always get approximately N buckets regardless of the data range, calculate the bucket size dynamically:

Always ~50 buckets

```
DECLARE @bucket_count := 50

WITH raw_data AS (
  SELECT price, amount FROM trades
  WHERE symbol = 'BTC-USDT' AND timestamp IN today()
),
bucket_size AS (
  SELECT (max(price) - min(price)) / (@bucket_count - 1) AS bucket_size FROM raw_data
)
SELECT
  floor(price / bucket_size) * bucket_size AS price_bin,
  round(sum(amount), 2) AS volume
FROM raw_data CROSS JOIN bucket_size
GROUP BY 1
ORDER BY 1;
```

Run on demo.questdb.com

This calculates `(max - min) / 49` to create 50 evenly distributed buckets. The `CROSS JOIN` makes the calculated bucket_size available to each row.

> **TIP: If there are fewer distinct values than requested buckets, or if some buckets have no data, you'll get fewer than 50 results.**

Solution 3: Logarithmic buckets For data spanning multiple orders of magnitude:

Logarithmic buckets for wide value ranges

```
SELECT
  power(10, floor(log(amount))) AS bucket,
  count(*) AS count
FROM trades
WHERE symbol = 'BTC-USDT'
  AND amount > 0.000001 -- optional. Just adding here for easier visualization
  AND timestamp IN today()
GROUP BY bucket
ORDER BY bucket;
```

Run on demo.questdb.com

Each bucket covers one order of magnitude (0.001-0.01, 0.01-0.1, 0.1-1.0, etc.).

Solution 4: Manual buckets For simple categorical grouping:

Manual category buckets

```
SELECT
  CASE
    WHEN amount < 0.01 THEN 'micro'
    WHEN amount < 0.1 THEN 'small'
    WHEN amount < 1.0 THEN 'medium'
    ELSE 'large'
  END AS bucket,
  count(*) AS count
FROM trades
WHERE symbol = 'BTC-USDT' AND timestamp IN today()
GROUP BY bucket;
```

Run on demo.questdb.com

Time-series histogram Track distribution changes over time by combining with `SAMPLE BY`:

Hourly histogram evolution

```
DECLARE @bucket_size := 0.5
SELECT
  timestamp,
  floor(amount / @bucket_size) * @bucket_size AS bucket,
  count(*) AS count
FROM trades
WHERE symbol = 'BTC-USDT' AND timestamp IN today()
SAMPLE BY 1h
ORDER BY timestamp, bucket;
```

Run on demo.questdb.com

View this recipe online

Create arrays from string literals

Cast string literals to array types for use with functions that accept array parameters.

Solution To cast an array from a string you need to cast to `double[]` for a vector, or to `double[][]` for a two-dimensional array. You can just keep adding brackets for as many dimensions as the literal has.

This query shows how to convert a string literal into an array, even when there are new lines:

Cast string to array

```
SELECT CAST('[
  [ 1.0, 2.0, 3.0 ],
  [
    4.0,
    5.0,
    6.0
  ]
]' AS double[][]),
cast('[[1,2,3],[4,5,6]]' as double[][]);
```

Run on demo.questdb.com

Note if you add the wrong number of brackets (for example, in this case if you try casting to `double[]` or `double[][][][]`), it will not error, but will instead convert as null.

View this recipe online

Integrations

Collect OPC-UA data with Telegraf in dense format

Configure Telegraf to collect OPC-UA industrial automation data and insert it into QuestDB in a dense format. By default, Telegraf creates one row per metric with sparse columns, but for QuestDB it's more efficient to merge all metrics from the same timestamp into a single dense row.

Problem: Sparse data format

When using Telegraf's OPC-UA input plugin with the default configuration, each metric value generates a separate row. Even when multiple metrics are collected at the same timestamp, they arrive as individual sparse rows:

Sparse format (inefficient):

timestamp	ServerLoad	ServerRAM	ServerIO
2024-01-15T10:00:00.000000Z	45.2	NULL	NULL
2024-01-15T10:00:00.000000Z	NULL	8192.0	NULL
2024-01-15T10:00:00.000000Z	NULL	NULL	1250.5

This wastes storage space and makes queries more complex.

Dense format (efficient):

timestamp	ServerLoad	ServerRAM	ServerIO
2024-01-15T10:00:00.000000Z	45.2	8192.0	1250.5

Solution: Use Telegraf's merge aggregator

Configure Telegraf to merge metrics with matching timestamps and tags before sending to QuestDB. This requires two key changes:

1. Add a common tag to all metrics
2. Use the `merge` aggregator to combine rows

Complete configuration

```
[agent]
  omit_hostname = true

## OPC-UA Input Plugin
[[inputs.opcua]]
  endpoint = "${OPCUA_ENDPOINT}"
  connect_timeout = "30s"
  request_timeout = "30s"
  security_policy = "None"
  security_mode = "None"
  auth_method = "Anonymous"
  name_override = "${METRICS_TABLE_NAME}"

  [[inputs.opcua.nodes]]
    name = "ServerLoad"
    namespace = "2"
    identifier_type = "s"
```

```
    identifier = "Server/Load"
    default_tags = { source="opcua_merge" }

  [[inputs.opcua.nodes]]
    name = "ServerRAM"
    namespace = "2"
    identifier_type = "s"
    identifier = "Server/RAM"
    default_tags = { source="opcua_merge" }

  [[inputs.opcua.nodes]]
    name = "ServerIO"
    namespace = "2"
    identifier_type = "s"
    identifier = "Server/IO"
    default_tags = { source="opcua_merge" }

## Merge Aggregator
[[aggregators.merge]]
  drop_original = true
  tags = ["source"]

## QuestDB Output via ILP
[[outputs.influxdb_v2]]
  urls = ["${QUESTDB_HTTP_ENDPOINT}"]
  token = "${QUESTDB_HTTP_TOKEN}"
  content_encoding = "identity"
```

Key configuration elements 1. Common tag

```
default_tags = { source="opcua_merge" }
```

Adds the same tag value (`source="opcua_merge"`) to all metrics. The merge aggregator uses this to identify which metrics should be combined.

2. Merge aggregator

```
[[aggregators.merge]]
  drop_original = true
  tags = ["source"]
```

- `drop_original = true`: Discards the original sparse rows after merging
- `tags = ["source"]`: Merges metrics with matching `source` tag values and the same timestamp

3. QuestDB output

```
[[outputs.influxdb_v2]]
  urls = ["${QUESTDB_HTTP_ENDPOINT}"]
  content_encoding = "identity"
```

- Uses the InfluxDB Line Protocol (ILP) over HTTP
- `content_encoding = "identity"`: Disables gzip compression (QuestDB doesn't require it)

How it works

The data flow is:

1. **OPC-UA server** → Telegraf collects metrics
2. **Telegraf input** → Creates separate rows for each metric with the `source="opcua_merge"` tag
3. **Merge aggregator** → Combines rows with matching timestamp + `source` tag
4. **QuestDB output** → Sends merged dense rows via ILP

Merging logic The merge aggregator combines metrics when: - **Timestamps match**: Metrics collected at the same moment - **Tags match**: All specified tags (in this case, `source`) have the same values

If metrics have different timestamps or tag values, they won't be merged.

Handling tag conflicts

If your OPC-UA nodes have additional tags with **different** values, those tags will prevent merging. Solutions:

Remove conflicting tags Use the `override` processor to remove unwanted tags:

```
[[processors.override]]
  [processors.override.tags]
    node_id = ""  # Removes the 'node_id' tag
    namespace = ""  # Removes the 'namespace' tag
```

Convert tags to fields Use the `converter` processor to convert tags to fields (fields don't affect merging):

```
[[processors.converter]]
  [processors.converter.tags]
    string = ["node_id", "namespace"]
```

This converts the tags to string fields, which won't interfere with the merge aggregator.

Remove the common tag after merging If you don't want the `source` tag in your final QuestDB table:

```
## Place this AFTER the merge aggregator
[[processors.override]]
  [processors.override.tags]
    source = ""  # Removes the 'source' tag
```

Environment variables

Use environment variables for sensitive configuration:

```
export OPCUA_ENDPOINT="opc.tcp://your-opcua-server:4840"
export METRICS_TABLE_NAME="industrial_metrics"
export QUESTDB_HTTP_ENDPOINT="http://questdb-host:9000"
export QUESTDB_HTTP_TOKEN="your_token_here"
```

Alternatively, use a `.env` file:

```
## .env file
OPCUA_ENDPOINT=opc.tcp://localhost:4840
METRICS_TABLE_NAME=opcua_metrics
QUESTDB_HTTP_ENDPOINT=http://localhost:9000
QUESTDB_HTTP_TOKEN=
```

Then start Telegraf with:

```
telegraf --config telegraf.conf
```

Verification

Query QuestDB to verify the data format:

```
SELECT * FROM opcua_metrics
ORDER BY timestamp DESC
LIMIT 10;
```

Expected: Dense rows with all metrics populated:

timestamp	source	ServerLoad	ServerRAM	ServerIO
2024-01-15T10:05:00.000000Z	opcua_merge	47.8	8256.0	1305.2
2024-01-15T10:04:00.000000Z	opcua_merge	45.2	8192.0	1250.5

Problem: Sparse rows with NULL values:

timestamp	source	ServerLoad	ServerRAM	ServerIO
2024-01-15T10:05:00.000000Z	opcua_merge	47.8	NULL	NULL
2024-01-15T10:05:00.000000Z	opcua_merge	NULL	8256.0	NULL

If you see sparse rows, check: - All nodes have the same `default_tags` - The merge aggregator is configured correctly - Timestamps are identical (not just close)

Alternative: TCP output

For higher throughput, use TCP instead of HTTP:

```
[[outputs.socket_writer]]
  address = "tcp://questdb-host:9009"
```

Differences: - **TCP**: Higher throughput, no acknowledgments, potential data loss on connection failure - **HTTP**: Reliable delivery, acknowledgments, slightly lower throughput

Choose TCP when: - You need maximum performance - Occasional data loss is acceptable - You're on a reliable local network

Choose HTTP when: - Data integrity is critical - You need error feedback - You're sending over the internet

Multiple OPC-UA sources

To collect from multiple OPC-UA servers into separate tables:

```
## Server 1
[[inputs.opcua]]
  endpoint = "opc.tcp://server1:4840"
  name_override = "server1_metrics"
  [[inputs.opcua.nodes]]
    name = "Temperature"
    namespace = "2"
    identifier_type = "s"
    identifier = "Sensor/Temp"
    default_tags = { source="server1" }

## Server 2
[[inputs.opcua]]
  endpoint = "opc.tcp://server2:4840"
  name_override = "server2_metrics"
  [[inputs.opcua.nodes]]
    name = "Pressure"
    namespace = "2"
    identifier_type = "s"
    identifier = "Sensor/Press"
    default_tags = { source="server2" }

## Merge by source tag
[[aggregators.merge]]
  drop_original = true
  tags = ["source"]
```

This creates two tables (`server1_metrics`, `server2_metrics`) with merged metrics from each server.

TIP: Performance Tuning

For high-frequency OPC-UA data: - Increase Telegraf's `flush_interval` to batch more data - Use `aggregators.merge.period` to specify merge window duration - Monitor QuestDB's ingestion rate and adjust accordingly

WARNING: Timestamp Precision

OPC-UA timestamps may have different precision than QuestDB expects. Ensure: - Telegraf agent precision matches your requirements (default: nanoseconds) - OPC-UA server timestamps are synchronized (use NTP) - Clock drift between systems is minimal

View this recipe online

Grafana

Query multiple tables dynamically in Grafana

Query multiple QuestDB tables dynamically in Grafana using dashboard variables. This is useful when you have many tables with identical schemas (e.g., sensor data, metrics from different sources) and want to visualize them together without hardcoding table names in your queries.

Problem: Visualize many similar tables You have 100+ tables with the same structure (e.g., `sensor_1`, `sensor_2`, ..., `sensor_n`) and want to: 1. Display data from all tables on a single Grafana chart 2. Avoid manually updating queries when tables are added or removed 3. Allow users to select which tables to visualize via dashboard controls

Solution: Use Grafana variables with dynamic SQL Create Grafana dashboard variables that query QuestDB for table names, then use string aggregation functions to build the SQL query dynamically.

Step 1: Get table names First, query QuestDB to get all relevant table names:

```
SELECT table_name FROM tables()
WHERE table_name LIKE 'sensor_%';
```

This returns a list of all tables matching the pattern.

Step 2: Create Grafana variables Create two dashboard variables to construct the dynamic query:

Variable 1: `$table_list` - Build the JOIN clause

```
WITH tbs AS (
  SELECT string_agg(table_name, ',') as names
  FROM tables()
  WHERE table_name LIKE 'sensor_%'
)
SELECT replace(names, ',', ' ASOF JOIN ') FROM tbs;
```

Output: `sensor_1 ASOF JOIN sensor_2 ASOF JOIN sensor_3 ASOF JOIN sensor_4`
This creates the table list with ASOF JOIN operators between them.

Variable 2: `$column_avgs` - Build the column list

```
SELECT string_agg(concat('avg(', table_name, '.value)'), ',') as columns
FROM tables()
WHERE table_name LIKE 'sensor_%';
```

Output: `avg(sensor_1.value),avg(sensor_2.value),avg(sensor_3.value),avg(sensor_4.value)`
This creates the column selection list with aggregation functions.

Step 3: Use variables in dashboard query Now reference these variables in your Grafana chart query:

```
SELECT sensor_1.timestamp, $column_avgs
FROM $table_list
SAMPLE BY 1s FROM $__fromTime TO $__toTime FILL(PREV);
```

When Grafana executes this query, it interpolates the variables:

```
SELECT sensor_1.timestamp, avg(sensor_1.value),avg(sensor_2.value),avg(sensor_3.value),avg(sensor_4.val
FROM sensor_1 ASOF JOIN sensor_2 ASOF JOIN sensor_3 ASOF JOIN sensor_4
SAMPLE BY 1s FROM cast(1571176800000000 as timestamp) TO cast(1571349600000000 as timestamp) FILL(PREV)
```

How it works The solution uses three key QuestDB features:

1. **`tables()` function**: Returns metadata about all tables in the database
2. **`string_agg()`**: Concatenates multiple rows into a single comma-separated string
3. **`replace()`**: Swaps commas for JOIN operators to build the FROM clause

Combined with Grafana's variable interpolation: - `$column_avgs`: Replaced with the aggregated column list - `$table_list`: Replaced with the joined table expression - `$__fromTime` / `$__toTime`: Grafana macros for the dashboard's time range

Understanding ASOF JOIN `ASOF JOIN` is ideal for time-series data with different update frequencies: - Joins tables on timestamp - For each row in the first table, finds the closest past timestamp in other tables - Works like a LEFT JOIN but with time-based matching

This ensures that even if tables update at different rates, you get a complete dataset with the most recent known value from each table.

Adapting the pattern **Filter by different patterns:**

```
-- Tables starting with "metrics_"
WHERE table_name LIKE 'metrics_%'

-- Tables matching a regex pattern
WHERE table_name ~ 'sensor_[0-9]+'

-- Exclude certain tables
WHERE table_name LIKE 'sensor_%'
  AND table_name NOT IN ('sensor_test', 'sensor_backup')
```

Programmatic alternative If you're not using Grafana, you can achieve the same result programmatically:

1. **Query for table names:**

   ```
   SELECT table_name FROM tables() WHERE table_name LIKE 'sensor_%';
   ```

2. **Build the query on the client side:**

   ```
   # Python example
   tables = ['sensor_1', 'sensor_2', 'sensor_3']

   # Build JOIN clause
   join_clause = ' ASOF JOIN '.join(tables)

   # Build column list
   columns = ','.join([f'avg({t}.value)' for t in tables])

   # Final query
   query = f"""
       SELECT {tables[0]}.timestamp, {columns}
       FROM {join_clause}
       SAMPLE BY 1s FILL(PREV)
   """
   ```

Handling different sampling intervals When tables have different update frequencies, use FILL to handle gaps:

```
-- Fill with previous value (holds last known value)
SAMPLE BY 1s FILL(PREV)

-- Fill with linear interpolation
SAMPLE BY 1s FILL(LINEAR)

-- Fill with NULL (show actual gaps)
SAMPLE BY 1s FILL(NULL)

-- Fill with zero
SAMPLE BY 1s FILL(0)
```

Choose based on your data: - **PREV**: Best for metrics that persist (temperatures, prices, statuses) - **LINEAR**: Best for continuous values that change smoothly - **NULL**: Best when you want to see actual data gaps - **0 or constant**: Best for counting or rate metrics

TIP: Performance Optimization

Joining many tables can be expensive. To improve performance: - Use `SAMPLE BY` to reduce the number of rows - Add timestamp filters early in the query - Consider pre-aggregating data into a single table for frequently-accessed views - Limit the number of tables joined (split into multiple charts if needed)

WARNING: Table Schema Consistency

This pattern assumes all tables have identical schemas. If schemas differ: - The query will fail at runtime - You'll need to handle missing columns explicitly - Consider using separate queries for tables with different structures

View this recipe online

Configure read-only user for Grafana

Configure a dedicated read-only user for Grafana to improve security by preventing accidental data modifications through dashboards. This allows you to maintain separate credentials for visualization (read-only) and administration (full access), following the principle of least privilege.

> **NOTE: QuestDB Enterprise**
>
> For QuestDB Enterprise, use the comprehensive Role-Based Access Control (RBAC) system to create granular user permissions and roles. The configuration below applies to QuestDB Open Source.

Problem: Separate read and write access You want to: 1. Connect Grafana with read-only credentials 2. Prevent accidental `INSERT`, `UPDATE`, `DELETE`, or `DROP` operations from dashboards 3. Still be able to execute DDL statements (`CREATE TABLE`, etc.) from the QuestDB web console

However, QuestDB's PostgreSQL wire protocol doesn't support standard PostgreSQL user management commands like `CREATE USER` or `GRANT`.

Solution: Enable the read-only user QuestDB Open Source supports a built-in read-only user that can be enabled via configuration. This gives you two users: - **Admin user** (default: `admin`): Full access for DDL and DML operations - **Read-only user** (default: `user`): Query-only access for dashboards

Configuration Add these settings to your `server.conf` file or set them as environment variables:

Via server.conf:

```
### Enable the read-only user
pg.readonly.user.enabled=true

### Optional: Customize username (default is "user")
pg.readonly.user=grafana_reader

### Optional: Customize password (default is "quest")
pg.readonly.password=secure_password_here
```

Via environment variables:

```
export QDB_PG_READONLY_USER_ENABLED=true
export QDB_PG_READONLY_USER=grafana_reader
export QDB_PG_READONLY_PASSWORD=secure_password_here
```

Via Docker:

```
docker run \
  -p 9000:9000 -p 8812:8812 \
  -e QDB_PG_READONLY_USER_ENABLED=true \
  -e QDB_PG_READONLY_USER=grafana_reader \
  -e QDB_PG_READONLY_PASSWORD=secure_password_here \
  questdb/questdb:latest
```

Using the read-only user After enabling, you have two separate users:

Admin user (web console): - Username: `admin` (default) - Password: `quest` (default) - Permissions: Full access - `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `CREATE`, `DROP`, `ALTER` - Use for: QuestDB web console, administrative tasks, schema changes

Read-only user (Grafana): - Username: `grafana_reader` (or whatever you configured) - Password: `secure_password_here` (or whatever you configured) - Permissions: `SELECT` queries only - Use for: Grafana dashboards, monitoring tools, analytics applications

View this recipe online

Grafana variable dropdown with name and value

Create Grafana variable dropdowns where the displayed label differs from the value used in queries. This is useful when you want to show user-friendly names in the dropdown while using different values (like IDs, prices, or technical identifiers) in your actual SQL queries.

Problem: Separate display and query values You want a Grafana variable dropdown that: - **Displays:** Readable labels like `"BTC-USDT"`, `"ETH-USDT"`, `"SOL-USDT"` - **Uses in queries:** Different values like prices (`37779.62`, `2615.54`, `98.23`) or IDs

For example, with this query result:

symbol	price
BTC-USDT	37779.62
ETH-USDT	2615.54
SOL-USDT	98.23

You want the dropdown to show `"BTC-USDT"` but use `37779.62` in your queries.

Solution: Use regex variable filters When using the QuestDB data source plugin, you can use Grafana's regex variable filters to parse a concatenated string into separate `text` and `value` fields.

Step 1: Concatenate columns in query First, combine both columns into a single string with a separator that doesn't appear in your data:

```
WITH t AS (
  SELECT symbol, first(price) as price
  FROM trades
  WHERE symbol LIKE '%BTC%'
)
SELECT concat(symbol, '#', price) FROM t;
```

Query results:

```
DOGE-BTC#0.00000204
ETH-BTC#0.05551
BTC-USDT#37779.62
SOL-BTC#0.0015282
MATIC-BTC#0.00002074
BTC-USDC#60511.1
```

Each row is now a single string with symbol and price separated by `#`.

Step 2: Apply regex filter in Grafana variable In your Grafana variable configuration:
Query:

```
WITH t AS (
  SELECT symbol, first(price) as price
  FROM trades
  WHERE symbol LIKE '%BTC%'
)
SELECT concat(symbol, '#', price) FROM t;
```

Regex Filter:

```
/(?<text>[^#]+)#(?<value>.*)/
```

This regex pattern: - `(?<text>[^#]+)`: Captures everything before `#` into the `text` group (the display label) - `#`: Matches the separator - `(?<value>.*)`: Captures everything after `#` into the `value` group (the query value)

Step 3: Use variable in queries Now you can reference the variable in your dashboard queries:

```
SELECT timestamp, price
FROM trades
WHERE price = $your_variable_name
  AND timestamp >= $__fromTime
  AND timestamp <= $__toTime;
```

When a user selects "BTC-USDT" from the dropdown, Grafana will substitute the corresponding price value (`37779.62`) into the query.

How it works Grafana's regex filter with named capture groups enables the separation:

1. **Named capture groups**: `(?<text>...)` and `(?<value>...)` tell Grafana which parts to use
2. **`text` group**: Becomes the visible label in the dropdown
3. **`value` group**: Becomes the interpolated value in queries
4. **Pattern matching**: The regex must match the entire string returned by your query

Regex pattern breakdown

```
/(?<text>[^#]+)#(?<value>.*)/
```

- `/`: Regex delimiters
- `(?<text>...)`: Named capture group called "text"
- `[^#]+`: One or more characters that are NOT `#` (greedy match)
- `#`: Literal separator character
- `(?<value>.*)`: Named capture group called "value"
- `.*`: Zero or more characters of any type (captures rest of string)

Choosing a separator Pick a separator that **never** appears in your data:

Good separators: - `#` - Uncommon in most data - `|` - Clear visual separator - `::` - Two characters, unlikely to appear - `~` - Rarely used in trading symbols or prices - `^^^` - Multi-character separator for extra safety

Bad separators: - `–` - Common in trading pairs (BTC-USDT) - `.` - Common in decimal numbers - `,` - Common in CSV-like data - Space - Can cause parsing issues

Alternative patterns

Multiple data fields If you need more than two fields, use additional separators:

```
SELECT concat(symbol, '#', price, '#', volume) FROM trades;
```

```
/(?<text>[^#]+)#(?<value>[^#]+)#(?<extra>.*)/
```

Now you have three captured groups, though Grafana's variable system typically only uses `text` and `value`.

Numeric IDs with descriptions Common pattern for entity selection:

```
SELECT concat(name, '#', id) FROM users;

/(?<text>[^#]+)#(?<value>\d+)/
```

Output in dropdown: User sees "John Doe", query uses `42`.

Escaping special characters If your data contains regex special characters, escape them in the pattern:

```
-- If data contains parentheses
SELECT concat(name, ' (', id, ')', '#', id) FROM users;
-- Result: "John Doe (42)#42"

/(?<text>.*?)#(?<value>\d+)/
```

PostgreSQL data source alternative If using the PostgreSQL data source (instead of the QuestDB plugin), you can use special column aliases:

```
SELECT
  symbol AS __text,
  price AS __value
FROM trades
WHERE symbol LIKE '%BTC%';
```

The PostgreSQL data source recognizes `__text` and `__value` as special column names for dropdown variables.

Note: This works with the PostgreSQL data source plugin pointing to QuestDB, but NOT with the native QuestDB data source plugin.

Adapting the pattern **Different filter conditions:**

```
-- Filter by time range
WHERE timestamp IN yesterday()

-- Filter by multiple criteria
WHERE symbol LIKE '%USDT' AND price > 1000

-- Dynamic filter using another variable
WHERE symbol LIKE concat('%', $base_currency, '%')
```

Sorting the dropdown:

```
-- Sort alphabetically by symbol
SELECT concat(symbol, '#', price) FROM trades
ORDER BY symbol;

-- Sort by price (highest first)
SELECT concat(symbol, '#', price) FROM trades
ORDER BY price DESC;

-- Sort by volume
WITH t AS (
  SELECT symbol, first(price) as price, sum(amount) as volume
  FROM trades
```

```
  GROUP BY symbol
)
SELECT concat(symbol, '#', price) FROM t
ORDER BY volume DESC;
```

Include additional context in label:

```
-- Show symbol and volume in the label
SELECT concat(symbol, ' (Vol: ', round(sum(amount), 2), ')', '#', first(price))
FROM trades
GROUP BY symbol;
```

Result: "BTC-USDT (Vol: 1234.56)#37779.62"

Troubleshooting **Dropdown shows concatenated string:** - Verify the regex pattern is correct - Check that the regex delimiters are `/.../` (forward slashes) - Ensure named capture groups are spelled correctly: `(?<text>...)` and `(?<value>...)`

Variable not interpolating in queries: - Verify you're using `$variable_name` syntax in queries - Check that the variable is defined at the dashboard level - Test the query manually with a hardcoded value

Regex not matching: - Test your regex pattern with a regex tester (regex101.com) - Verify your separator doesn't appear in the data itself - Check for trailing whitespace in query results

Dropdown is empty: - Verify the query returns data - Check that QuestDB is accessible from Grafana - Review Grafana logs for error messages

TIP: Multi-Select Variables

This pattern works with multi-select variables too. Enable "Multi-value" in the variable configuration, and users can select multiple options. Use `IN ($variable)` in your queries to handle multiple selected values.

TIP: Variable Preview

Grafana shows a preview of what the dropdown will look like when you configure the regex filter. Use this to verify your pattern is working correctly before applying it.

View this recipe online

Overlay two time series with time shift

Compare yesterday's data against today's data on the same Grafana chart by overlaying them.

Problem You have a query with Grafana's `timeshift` set to `1d/d` to display yesterday's data. You want to overlay today's data on the same chart, starting from scratch each day, so you can compare the shapes of both time series.

Solution Leave the timeshift as `1d/d` to cover yesterday, and add a new query to the same chart. In this new query, filter for timestamp plus 1 day to cover today's datapoints, then shift them back by 1 day for display.

Query 1 (Yesterday's data):

```
DECLARE
  @symbol := 'BTC-USDT'
WITH sampled AS (
    SELECT
          timestamp,  symbol,
          volume AS volume,
          ((open+close)/2) * volume AS traded_value
     FROM trades_OHLC_15m
     WHERE $__timeFilter(timestamp)
     AND symbol = @symbol
), cumulative AS (
     SELECT timestamp, symbol,
           SUM(traded_value)
                OVER (ORDER BY timestamp) AS cumulative_value,
           SUM(volume)
                OVER (ORDER BY timestamp) AS cumulative_volume
     FROM sampled
)
SELECT timestamp as time, cumulative_value/cumulative_volume AS vwap_yesterday FROM cumulative;
```

Query 2 (Today's data, shifted back):

```
DECLARE
  @symbol := 'BTC-USDT'
WITH sampled AS (
    SELECT
          timestamp,  symbol,
          volume AS volume,
          ((open+close)/2) * volume AS traded_value
     FROM trades_OHLC_15m
     WHERE timestamp BETWEEN dateadd('d',1,$__unixEpochFrom()*1000000)
       AND dateadd('d',1,$__unixEpochTo() * 1000000)
     AND symbol = @symbol
), cumulative AS (
     SELECT timestamp, symbol,
           SUM(traded_value)
                OVER (ORDER BY timestamp) AS cumulative_value,
           SUM(volume)
                OVER (ORDER BY timestamp) AS cumulative_volume
     FROM sampled
)
SELECT dateadd('d',-1,timestamp) as time, cumulative_value/cumulative_volume AS vwap_today FROM cumulat
```

Note: This example uses `$__unixEpochFrom()` and `$__unixEpochTo()` macros from the PostgreSQL Grafana plugin. When using the QuestDB plugin, the equivalent macros are `$__fromTime` and `$__toTime` and don't need epoch conversion as those are native timestamps.

This creates an overlay chart where yesterday's and today's data align on the same time axis, allowing direct comparison.

View this recipe online

Programmatic Access

Configure TLS certificate authorities

Configure TLS certificate authority (CA) validation when connecting QuestDB clients to TLS-enabled instances.

Problem

You are using a QuestDB client (Rust, Python, C++, etc.) to insert data. It works when using QuestDB without TLS, but when you enable TLS on your QuestDB instance using a self-signed certificate, you get an error of "certificate unknown".

When using the PostgreSQL wire interface, you can insert data passing `sslmode=require`, and it works, so you can discard any problems with QuestDB recognizing the certificate. But you need to figure out the equivalent for your ILP client.

Solution: Configure TLS CA

QuestDB clients support the `tls_ca` parameter, which has multiple values to configure certificate authority validation:

Option 1: Use WebPKI and OS certificate roots (recommended for production) If you want to accept both the webpki-root certificates plus whatever you have on the OS, pass `tls_ca=webpki_and_os_roots`:

```
https::addr=localhost:9000;username=admin;password=quest;tls_ca=webpki_and_os_roots;
```

This will work with certificates signed by standard certificate authorities.

Option 2: Use a custom PEM file Point to a PEM-encoded certificate file for self-signed or custom CA certificates:

```
https::addr=localhost:9000;username=admin;password=quest;tls_ca=pem_file;tls_roots=/path/to/ce
```

This is useful for self-signed certificates or internal CAs.

Option 3: Skip verification (development only) For development environments with self-signed certificates, you might be tempted to disable verification by passing `tls_verify=unsafe_off`:

```
https::addr=localhost:9000;username=admin;password=quest;tls_verify=unsafe_off;
```

> **DANGER: This is a very bad idea for production and should only be used for testing on a development environment with a self-signed certificate. It disables all certificate validation.**

Note: Some clients require enabling an optional feature (like `insecure-skip-verify` in Rust) before the `tls_verify=unsafe_off` parameter will work. Check your client's documentation for details.

Available tls_ca values

Value	Description
`webpki_roots`	Mozilla's WebPKI root certificates only
`os_roots`	Operating system certificate store only
`webpki_and_os_roots`	Both WebPKI and OS roots (recommended)
`pem_file`	Load from a PEM file (requires `tls_roots` parameter)

Example: Rust client

```rust
use questdb::ingress::{Sender, SenderBuilder};

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let sender = SenderBuilder::new("https", "localhost", 9000)?
        .username("admin")?
        .password("quest")?
        .tls_ca("webpki_and_os_roots")?  // Use standard CAs
        .build()
        .await?;

    // Use sender...

    sender.close().await?;
    Ok(())
}
```

For self-signed certificates with a PEM file:

```rust
let sender = SenderBuilder::new("https", "localhost", 9000)?
    .username("admin")?
    .password("quest")?
    .tls_ca("pem_file")?
    .tls_roots("/path/to/questdb.crt")?
    .build()
    .await?;
```

The examples are in Rust but the concepts are similar in other languages. Check the documentation for your specific client.

View this recipe online

PHP

Insert data from PHP using ILP

QuestDB doesn't maintain an official PHP library, but since the ILP (InfluxDB Line Protocol) is text-based, you can easily send your data using PHP's built-in HTTP or socket functions, or use the official InfluxDB PHP client library.

Available approaches This guide covers three methods for sending ILP data to QuestDB from PHP:

1. **HTTP with cURL** (recommended for most use cases)
   - Full control over ILP formatting and timestamps
   - No external dependencies beyond PHP's built-in cURL
   - Requires manual ILP string construction
2. **InfluxDB v2 PHP Client** (easiest to use)
   - Clean Point builder API
   - Automatic batching and error handling
   - **Limitation:** Cannot use custom timestamps with QuestDB (must use server timestamps)
   - Requires Composer packages: `influxdata/influxdb-client-php` and `guzzlehttp/guzzle`
3. **TCP Socket** (highest throughput)
   - Best performance for high-volume scenarios
   - No acknowledgments - data loss possible
   - Manual implementation required

ILP protocol overview The ILP protocol allows you to send data to QuestDB using a simple line-based text format:

```
table_name,comma_separated_symbols comma_separated_non_symbols optional_timestamp\n
```

Each line represents one row of data. For example, these two lines are well-formed ILP messages:

```
readings,city=London,make=Omron temperature=23.5,humidity=0.343 1465839830100400000\n
readings,city=Bristol,make=Honeywell temperature=23.2,humidity=0.443\n
```

The format consists of: - **Table name**: The target table for the data - **Symbols** (tags): Comma-separated key-value pairs for indexed categorical data - **Columns** (fields): Space-separated, then comma-separated key-value pairs for numerical or string data - **Timestamp** (optional): Nanosecond-precision timestamp; if omitted, QuestDB uses server time

For complete ILP specification, see the ILP reference documentation.

ILP over HTTP QuestDB supports ILP data via HTTP or TCP. **HTTP is the recommended approach** for most use cases as it provides better reliability and easier debugging.

To send data via HTTP: 1. Send a POST request to `http://localhost:9000/write` (or your QuestDB instance endpoint) 2. Set `Content-Type: text/plain` header 3. Include ILP-formatted rows in the request body 4. For higher throughput, batch multiple rows in a single request

HTTP buffering example The following PHP class provides buffered insertion with automatic flushing based on either row count or elapsed time:

"'php title="Buffered ILP insertion via HTTP" bufferSize = $bufferSize; $this->flushInterval = $flushInterval; $this->lastFlushTime = time(); }

```
public function __destruct() {
    // Attempt to flush any remaining data when script is terminating
    $this->flush();
}

public function insertRow($tableName, $symbols, $columns, $timestamp = null) {
    $row = $this->formatRow($tableName, $symbols, $columns, $timestamp);
    $this->buffer[] = $row;
    $this->checkFlushConditions();
}

private function formatRow($tableName, $symbols, $columns, $timestamp) {
    $escape = function($value) {
        return str_replace([' ', ',', "\n"], ['\ ', '\,', '\n'], $value);
    };

    $symbolString = implode(',', array_map(
        function($k, $v) use ($escape) { return "$k={$escape($v)}"; },
        array_keys($symbols), $symbols
    ));

    $columnString = implode(',', array_map(
        function($k, $v) use ($escape) { return "$k={$escape($v)}"; },
        array_keys($columns), $columns
    ));

    // Check if timestamp is provided
    $timestampPart = is_null($timestamp) ? '' : " $timestamp";

    return "$tableName,$symbolString $columnString$timestampPart";
}

private function checkFlushConditions() {
    if (count($this->buffer) >= $this->bufferSize || (time() - $this->lastFlushTime) >= $this-
        $this->flush();
    }
}

private function flush() {
    if (empty($this->buffer)) {
        return; // Nothing to flush
    }
    $data = implode("\n", $this->buffer);
    $this->buffer = [];
    $this->lastFlushTime = time();

    $ch = curl_init();
```

```
    curl_setopt($ch, CURLOPT_URL, $this->endpoint);
    curl_setopt($ch, CURLOPT_POST, true);
    curl_setopt($ch, CURLOPT_POSTFIELDS, $data);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    curl_setopt($ch, CURLOPT_HTTPHEADER, ['Content-Type: text/plain']);
    curl_exec($ch);
    curl_close($ch);
}
```

}
// Usage example: $inserter = new DataInserter(10, 30);
// Inserting rows for London $inserter->insertRow("test_readings", ["city" => "London", "make" => "Omron"], ["temperature" => 23.5, "humidity" => 0.343], "1650573480100400000");inserter->insertRow("test_readings", ["city" => "London", "make" => "Sony"], ["temperature" => 21.0, "humidity" => 0.310]); $inserter->insertRow("test_readings", ["city" => "London", "make" => "Philips"], ["temperature" => 22.5, "humidity" => 0.333], "1650573480100500000");inserter->insertRow("test_readings", ["city" => "London", "make" => "Samsung"], ["temperature" => 24.0, "humidity" => 0.350]);
// Inserting rows for Madrid $inserter->insertRow("test_readings", ["city" => "Madrid", "make" => "Omron"], ["temperature" => 25.5, "humidity" => 0.360], "1650573480100600000");inserter->insertRow("test_readings", ["city" => "Madrid", "make" => "Sony"], ["temperature" => 23.0, "humidity" => 0.340]); $inserter->insertRow("test_readings", ["city" => "Madrid", "make" => "Philips"], ["temperature" => 26.0, "humidity" => 0.370], "1650573480100700000");inserter->insertRow("test_readings", ["city" => "Madrid", "make" => "Samsung"], ["temperature" => 22.0, "humidity" => 0.355]);
// Inserting rows for New York $inserter->insertRow("test_readings", ["city" => "NewYork", "make" => "Omron"], ["temperature" => 20.5, "humidity" => 0.330], "1650573480100800000");inserter->insertRow("test_readings", ["city" => "New York", "make" => "Sony"], ["temperature" => 19.0, "humidity" => 0.320]); $inserter->insertRow("test_readings", ["city" => "NewYork", "make" => "Philips"], ["temperature" => 21.0, "humidity" => 0.340], "1650573480100900000");inserter->insertRow("test_readings", ["city" => "New York", "make" => "Samsung"], ["temperature" => 18.5, "humidity" => 0.335]); ?>

```
This class:
- Buffers rows until either 10 rows are accumulated or 30 seconds have elapsed
- Properly escapes special characters (spaces, commas, newlines) in values
- Automatically flushes remaining data when the script terminates
- Uses cURL for HTTP communication

> **TIP: For production use, consider adding error handling to check the HTTP response status
>
>


#### Using the InfluxDB v2 PHP client

Another approach is to use the official [InfluxDB PHP client library](https://github.com/influ

##### Installation

Install the required packages via Composer:
```

```bash
composer require influxdata/influxdb-client-php guzzlehttp/guzzle
```

Required dependencies: - `influxdata/influxdb-client-php` - The InfluxDB v2 PHP client library - `guzzlehttp/guzzle` - A PSR-18 compatible HTTP client (required by the InfluxDB client)

> **INFO: Alternative HTTP Clients**
>
> The InfluxDB client requires a PSR-18 compatible HTTP client. While we recommend Guzzle, you can use alternatives like `php-http/guzzle7-adapter` or `symfony/http-client` if preferred.

Configuration When using the InfluxDB client with QuestDB:

- **URL**: Use your QuestDB HTTP endpoint (default: `http://localhost:9000`)
- **Token**: Not required - can be left empty or use any string
- **Bucket**: Not required - can be any string (ignored by QuestDB)
- **Organization**: Not required - can be any string (ignored by QuestDB)

> **WARNING: Write API Only**
>
> QuestDB only supports the **InfluxDB v2 write API** when using this client. Query operations are not supported through the InfluxDB client - use QuestDB's PostgreSQL wire protocol or REST API for queries instead.

Example code "'php title="Using InfluxDB v2 PHP client with QuestDB" "http://localhost:9000", "token" => " ", // Not required for QuestDB"bucket" => "default", // Not used by QuestDB "org" => "default", // Not used by QuestDB "precision" => WritePrecision::NS]);
$writeApi = $client->createWriteApi();
// Write points using the Point builder // Note: Omit ->time() to let QuestDB assign server timestamps $point = Point::measurement("readings") ->addTag("city", "London") ->addTag("make", "Omron") ->addField("temperature", 23.5) ->addField("humidity", 0.343);
$writeApi->write(point);
// Write multiple points $points = [Point::measurement("readings") ->addTag("city", "Madrid") ->addTag("make", "Sony") ->addField("temperature", 25.5) ->addField("humidity", 0.360),

```
Point::measurement("readings")
    ->addTag("city", "New York")
    ->addTag("make", "Philips")
    ->addField("temperature", 20.5)
    ->addField("humidity", 0.330)

];
```

$writeApi->write(points);
// Always close the client $client->close(); ?>

```
##### Benefits and limitations

The Point builder provides several advantages:
- **Automatic ILP formatting and escaping** - No need to manually construct ILP strings
- **Built-in error handling** - The client handles HTTP errors and retries
```

```
- **Batching support** - Automatically batches writes for better performance
- **Clean API** - Fluent Point builder interface is easy to use

> **WARNING: Timestamp Limitation**
>
> The InfluxDB PHP client **cannot be used with custom timestamps** when writing to QuestDB. W
>
> **Solution:** Always omit the `->time()` call and let QuestDB assign server-side timestamps
>
> **If you need client-side timestamps:** Use the raw HTTP cURL approach (documented above) wh


#### ILP over TCP socket

TCP over socket provides higher throughput but is less reliable than HTTP. The message format

Use TCP when:
- You need maximum ingestion throughput
- Your application can handle potential data loss on connection failures
- You're willing to implement your own connection management and error handling

##### TCP socket example

Here's a basic example using PHP's socket functions:

```php title="Send ILP data via TCP socket"
<?php
error_reporting(E_ALL);

/* Allow the script to hang around waiting for connections. */
set_time_limit(0);

/* Turn on implicit output flushing so we see what we're getting
 * as it comes in. */
ob_implicit_flush();

$address = 'localhost';
$port = 9009;

/* Create a TCP/IP socket. */
$socket = socket_create(AF_INET, SOCK_STREAM, SOL_TCP);
if ($socket === false) {
    echo "socket_create() failed: reason: " . socket_strerror(socket_last_error()) . "\n";
} else {
    echo "OK.\n";
}

echo "Attempting to connect to '$address' on port '$port'...";
$result = socket_connect($socket, $address, $port);
if ($result === false) {
    echo "socket_connect() failed.\nReason: ($result) " . socket_strerror(socket_last_error($s
```
```

```
} else {
    echo "OK.\n";
}

$row=utf8_encode("test_readings,city=London,make=Omron temperature=23.5,humidity=0.343 1465839
echo "$row";
socket_write($socket, $row);
echo "\n";
socket_close($socket);

?>
```

This basic example: - Connects to QuestDB's ILP port (default 9009) - Sends a single row of data - Closes the connection

For production use with TCP, you should: - Keep connections open and reuse them for multiple rows - Implement batching to reduce network overhead - Add proper error handling and reconnection logic - Consider using a connection pool for concurrent writes

> **WARNING: TCP Considerations**
>
> TCP ILP does not provide acknowledgments for successful writes. If the connection drops, you may lose data without notification. For critical data, use HTTP ILP instead.

Choosing the right approach

Feature	HTTP (cURL)	HTTP (InfluxDB Client)	TCP Socket
Reliability	High - responses indicate success/failure	High - responses indicate success/failure	Low - no acknowledgment
Throughput	Good	Good	Excellent
Error handling	Manual via cURL	Built-in via client library	Manual implementation required
Ease of use	Medium - manual ILP formatting	High - Point builder API	Low - manual everything
Custom timestamps	□ Full control	□ Must use server timestamps	□ Full control
Dependencies	None (cURL built-in)	`influxdb-client-phpguzzlehttp/guzzle`	None (sockets built-in)
Authentication	Standard HTTP auth	Standard HTTP auth	Limited options
Recommended for	Custom timestamps required	Ease of development, server timestamps acceptable	High-volume, loss-tolerant scenarios

View this recipe online

Ruby

Insert data from Ruby using ILP

Send time-series data from Ruby to QuestDB using the InfluxDB Line Protocol (ILP). While QuestDB doesn't maintain an official Ruby client, you can easily use the official InfluxDB Ruby gem to send data via ILP over HTTP, which QuestDB fully supports.

Available approaches Two methods for sending ILP data from Ruby:

1. **InfluxDB v2 Ruby Client** (recommended)
   - Official InfluxDB gem with clean API
   - Automatic batching and error handling
   - Compatible with QuestDB's ILP endpoint
   - Requires: `influxdb-client` gem
2. **TCP Socket** (for custom implementations)
   - Direct socket communication
   - Manual ILP message formatting
   - Higher throughput, no dependencies
   - Requires: Built-in Ruby socket library

Using the InfluxDB v2 Ruby client The InfluxDB v2 client provides a convenient Point builder API that works with QuestDB.

Installation

```
gem install influxdb-client
```

Or add to your `Gemfile`:

```
gem 'influxdb-client', '~> 3.1'
```

Example code

```
require 'influxdb-client'

### Create client
client = InfluxDB2::Client.new(
  'http://localhost:9000',
  'ignore-token',  # Token not required for QuestDB
  bucket: 'ignore-bucket',  # Bucket not used by QuestDB
  org: 'ignore-org',  # Organization not used by QuestDB
  precision: InfluxDB2::WritePrecision::NANOSECOND,
  use_ssl: false
)

write_api = client.create_write_api

### Write a single point
point = InfluxDB2::Point.new(name: 'readings')
  .add_tag('city', 'London')
  .add_tag('make', 'Omron')
  .add_field('temperature', 23.5)
  .add_field('humidity', 0.343)
```

```
write_api.write(data: point)

### Write multiple points
points = [
  InfluxDB2::Point.new(name: 'readings')
    .add_tag('city', 'Madrid')
    .add_tag('make', 'Sony')
    .add_field('temperature', 25.5)
    .add_field('humidity', 0.360),

  InfluxDB2::Point.new(name: 'readings')
    .add_tag('city', 'New York')
    .add_tag('make', 'Philips')
    .add_field('temperature', 20.5)
    .add_field('humidity', 0.330)
]

write_api.write(data: points)

### Always close the client
client.close!
```

Configuration notes When using the InfluxDB client with QuestDB:

- **`token`**: Not required - can be empty string or any value
- **`bucket`**: Ignored by QuestDB - can be any string
- **`org`**: Ignored by QuestDB - can be any string
- **`precision`**: Use `NANOSECOND` for compatibility (QuestDB's native precision)
- **`use_ssl`**: Set to `false` for local development, `true` for production with TLS

Data types The InfluxDB client automatically handles type conversions:

```
point = InfluxDB2::Point.new(name: 'measurements')
  .add_tag('sensor_id', '001')                    # SYMBOL in QuestDB
  .add_field('temperature', 23.5)                  # DOUBLE
  .add_field('humidity', 0.343)                    # DOUBLE
  .add_field('pressure', 1013)                     # LONG (integer)
  .add_field('status', 'active')                   # STRING
  .add_field('online', true)                       # BOOLEAN
```

TCP socket approach For maximum control and performance, send ILP messages directly via TCP sockets.

Basic TCP example

```
require 'socket'

HOST = 'localhost'
PORT = 9009

### Helper method to get current time in nanoseconds
def time_in_nsec
  now = Time.now
  return now.to_i * (10 ** 9) + now.nsec
end
```

```
begin
  s = TCPSocket.new(HOST, PORT)

  # Single record with timestamp
  s.puts "trades,symbol=BTC-USDT,side=buy price=37779.62,amount=0.5 #{time_in_nsec}\n"

  # Omitting timestamp - server assigns one
  s.puts "trades,symbol=ETH-USDT,side=sell price=2615.54,amount=1.2\n"

  # Multiple records (newline-delimited)
  s.puts "trades,symbol=SOL-USDT,side=buy price=98.23,amount=10.0\n" +
         "trades,symbol=BTC-USDT,side=sell price=37800.00,amount=0.3\n"

rescue SocketError => ex
  puts "Socket error: #{ex.inspect}"
ensure
  s.close if s
end
```

ILP message format The ILP format is:

```
table_name,tag1=value1,tag2=value2 field1=value1,field2=value2 timestamp\n
```

Breaking it down: - **Table name**: Target table (created automatically if doesn't exist) - **Tags** (symbols): Comma-separated key=value pairs for indexed categorical data - **Space separator**: Separates tags from fields - **Fields** (columns): Comma-separated key=value pairs for numerical or string data - **Space separator**: Separates fields from timestamp - **Timestamp** (optional): Nanosecond-precision timestamp; if omitted, server assigns

Example:

```
readings,city=London,make=Omron temperature=23.5,humidity=0.343 1465839830100400000\n
```

Escaping special characters ILP requires escaping for certain characters:

```
def escape_ilp(value)
  value.to_s
    .gsub(' ', '\\ ')     # Space
    .gsub(',', '\\,')     # Comma
    .gsub('=', '\\=')     # Equals
    .gsub("\n", '\\n')    # Newline
end

### Usage
tag_value = "London, UK"
escaped = escape_ilp(tag_value)  # "London\\, UK"

s.puts "readings,city=#{escaped} temperature=23.5\n"
```

Batching for performance Send multiple rows in a single TCP write:

```
require 'socket'

HOST = 'localhost'
PORT = 9009
```

```
def time_in_nsec
  now = Time.now
  return now.to_i * (10 ** 9) + now.nsec
end

begin
  s = TCPSocket.new(HOST, PORT)

  # Build batch of rows
  batch = []
  (1..1000).each do |i|
    timestamp = time_in_nsec + i * 1000000  # 1ms apart
    batch << "readings,sensor_id=#{i} value=#{rand(100.0)},status=\"ok\" #{timestamp}"
  end

  # Send entire batch at once
  s.puts batch.join("\n") + "\n"
  s.flush

rescue SocketError => ex
  puts "Socket error: #{ex.inspect}"
ensure
  s.close if s
end
```

Comparison: InfluxDB client vs TCP socket

Feature	InfluxDB Client	TCP Socket
Ease of use	High - Point builder API	Medium - Manual ILP formatting
Dependencies	Requires `influxdb-client` gem	None (stdlib only)
Error handling	Automatic with retries	Manual implementation
Batching	Automatic	Manual
Performance	Good	Excellent (direct TCP)
Type safety	Automatic type conversion	Manual string formatting
Reliability	HTTP with acknowledgments	No acknowledgments (fire and forget)
Escaping	Automatic	Manual implementation required
Recommended for	Most applications	High-throughput scenarios, custom needs

Best practices

Connection management InfluxDB Client:

```
### Reuse client for multiple writes
client = InfluxDB2::Client.new(...)
write_api = client.create_write_api

### ... perform many writes ...

client.close!  # Always close when done
```

TCP Socket:

```
### Keep connection open for multiple writes
socket = TCPSocket.new(HOST, PORT)

begin
  # ... send multiple batches ...
ensure
  socket.close if socket
end
```

Error handling **InfluxDB Client:**

```
begin
  write_api.write(data: points)
rescue InfluxDB2::InfluxError => e
  puts "Failed to write to QuestDB: #{e.message}"
  # Implement retry logic or logging
end
```

TCP Socket:

```
begin
  socket.puts(ilp_messages)
  socket.flush
rescue Errno::EPIPE, Errno::ECONNRESET => e
  puts "Connection lost: #{e.message}"
  # Reconnect and retry
rescue StandardError => e
  puts "Unexpected error: #{e.message}"
end
```

Timestamp generation Use nanosecond precision for maximum compatibility:

```
### Current time in nanoseconds
def current_nanos
  now = Time.now
  now.to_i * 1_000_000_000 + now.nsec
end

### Specific time to nanoseconds
def time_to_nanos(time)
  time.to_i * 1_000_000_000 + time.nsec
end

### Usage
timestamp = current_nanos
### or
timestamp = time_to_nanos(Time.parse("2024-09-05 14:30:00 UTC"))
```

Batching strategy For high-throughput scenarios:

```
BATCH_SIZE = 1000
FLUSH_INTERVAL = 5  # seconds

batch = []
last_flush = Time.now
```

```
data_stream.each do |record|
  batch << format_ilp_message(record)

  if batch.size >= BATCH_SIZE || (Time.now - last_flush) >= FLUSH_INTERVAL
    socket.puts batch.join("\n") + "\n"
    socket.flush
    batch.clear
    last_flush = Time.now
  end
end

### Flush remaining records
socket.puts batch.join("\n") + "\n" unless batch.empty?
```

TIP: Choosing an Approach

- **Use InfluxDB client** for most Ruby applications - it's easier, safer, and handles edge cases
- **Use TCP sockets** only when you need maximum throughput and can handle reliability concerns

WARNING: Data Loss with TCP

TCP ILP has no acknowledgments. If the connection drops, data may be lost silently. For critical data, use HTTP (via the InfluxDB client) which provides delivery confirmation.

View this recipe online

C++

Handle missing columns in C++ client

Send rows with missing or optional columns to QuestDB using the C++ client.

Problem In Python, you can handle missing columns easily with dictionaries:

```
{"price1": 10.0, "price2": 10.1}
```

And if price2 is not available:

```
{"price1": 10.0, "price2": None}
```

Which is equivalent to:

```
{"price1": 10.0}
```

You can pass the dict as the columns argument to `sender.rows` and it transparently sends the rows, with or without missing columns, to the server.

In C++, the buffer API requires explicit method calls:

```
buffer
    .table("trades")
    .symbol("symbol", "ETH-USD")
    .symbol("side", "sell")
    .column("price", 2615.54)
    .column("amount", 0.00044)
    .at(questdb::ingress::timestamp_nanos::now());

sender.flush(buffer);
```

How do you handle "ragged" rows with missing columns in C++?

Solution You need to call `at` at the end of the buffer so the data gets queued to be sent, but you can call `symbol` and `column` as many times as needed for each row, and you can do this conditionally.

The example below builds a vector with three rows, one of them with an empty column, then it iterates over the vector and checks if the optional `price` column is null. If it is, it skips invoking `column` for the buffer on that column.

```
#include <questdb/ingress/line_sender.hpp>
#include <iostream>
#include <chrono>
#include <vector>
#include <optional>
#include <string>

int main()
{
    try
    {
        auto sender = questdb::ingress::line_sender::from_conf(
            "http::addr=localhost:9000;username=admin;password=quest;retry_timeout=20000;");

        auto now = std::chrono::system_clock::now();
        auto duration = now.time_since_epoch();
        auto nanos = std::chrono::duration_cast<std::chrono::nanoseconds>(duration).count();
```

```
        struct Row {
            std::string symbol;
            std::string side;
            std::optional<double> price;
            double amount;
        };

        std::vector<Row> rows = {
            {"ETH-USD", "sell", 2615.54, 0.00044},
            {"BTC-USD", "sell", 39269.98, 0.001},
            {"SOL-USD", "sell", std::nullopt, 5.5} // Missing price
        };

        questdb::ingress::line_sender_buffer buffer;

        for (const auto& row : rows) {
            buffer.table("trades")
                .symbol("symbol", row.symbol)
                .symbol("side", row.side);

            if (row.price.has_value()) {
                buffer.column("price", row.price.value());
            }

            buffer.column("amount", row.amount)
                .at(questdb::ingress::timestamp_nanos(nanos));
        }

        sender.flush(buffer);
        sender.close();

        std::cout << "Data successfully sent!" << std::endl;
        return 0;
    }
    catch (const questdb::ingress::line_sender_error& err)
    {
        std::cerr << "Error running example: " << err.what() << std::endl;
        return 1;
    }
}
```

View this recipe online

Operations

Configure QuestDB with Docker Compose

You can override any QuestDB configuration parameter using environment variables in Docker Compose. This is useful for setting custom ports, authentication credentials, memory limits, and other operational settings without modifying configuration files.

Environment variable format

To override configuration parameters via environment variables:

1. **Prefix with `QDB_`**: Add `QDB_` before the parameter name
2. **Capitalize**: Convert to uppercase
3. **Replace dots with underscores**: Change `.` to _

For example: - `pg.user` becomes `QDB_PG_USER` - `pg.password` becomes `QDB_PG_PASSWORD` - `cairo.sql.copy.buffer.size` becomes `QDB_CAIRO_SQL_COPY_BUFFER_SIZE`

> **TIP: Keep sensitive configuration like passwords in a `.env` file and reference them in `docker-compose.yml`:**
>
> ```
> environment:
>  - QDB_PG_PASSWORD=${QUESTDB_PASSWORD}
> ```
>
> Then create a `.env` file:
>
> ```
> QUESTDB_PASSWORD=your_secure_password
> ```

Example: Custom PostgreSQL credentials

This Docker Compose file overrides the default PostgreSQL wire protocol credentials:

"'yaml title="docker-compose.yml - Override pg.user and pg.password" version: "3.9" services: questdb: image: questdb/questdb container_name: custom_questdb restart: always ports: - "8812:8812" - "9000:9000" - "9009:9009" - "9003:9003" extra_hosts: - "host.docker.internal:hos gateway" environment: - QDB_PG_USER=borat - QDB_PG_PASSWORD=clever_password volumes: - ./questdb/questdb_root:/var/lib/questdb/

```
This configuration:
- Sets PostgreSQL wire protocol username to `borat`
- Sets password to `clever_password`
- Persists data to `./questdb/questdb_root` on the host machine
- Exposes all QuestDB ports (web console, HTTP, ILP, PostgreSQL wire)
```

```
> **WARNING: Volume Permissions**
>
> If you encounter permission errors with mounted volumes, ensure the QuestDB container user h
```

```
### Custom data directory permissions
```

```yaml title="Run with specific user/group for volume permissions"
services:
  questdb:
    image: questdb/questdb
    user: "1000:1000"
    environment:
      - QDB_CAIRO_ROOT=/var/lib/questdb
    volumes:
      - ./questdb_data:/var/lib/questdb
```

Complete configuration reference

For a full list of available configuration parameters, see: - Server Configuration Reference - All configurable parameters with descriptions - Docker Deployment Guide - Docker-specific setup instructions

View this recipe online

Store QuestDB metrics in QuestDB

Store QuestDB's operational metrics in QuestDB itself by scraping Prometheus metrics using Telegraf.

Solution: Telegraf configuration

You could use Prometheus to scrape those metrics, but you can also use any server agent that understands the Prometheus format. It turns out Telegraf has input plugins for Prometheus and output plugins for QuestDB, so you can use it to get the metrics from the endpoint and insert them into a QuestDB table.

This is a `telegraf.conf` configuration which works (using default ports):

```
## Configuration for Telegraf agent
[agent]
  ## Default data collection interval for all inputs
  interval = "5s"
  omit_hostname = true
  precision = "1ms"
  flush_interval = "5s"

## -- INPUT PLUGINS ------------------------------------------------------- #
[[inputs.prometheus]]
  ## An array of urls to scrape metrics from.
  urls = ["http://questdb-origin:9003/metrics"]
  url_tag=""
  metric_version = 2 # all entries will be on a single table
  ignore_timestamp = false

## -- AGGREGATOR PLUGINS -------------------------------------------------- #
## Merge metrics into multifield metrics by series key
[[aggregators.merge]]
  ## If true, the original metric will be dropped by the
  ## aggregator and will not get sent to the output plugins.
  drop_original = true


## -- OUTPUT PLUGINS ------------------------------------------------------ #
[[outputs.socket_writer]]
  # Write metrics to a local QuestDB instance over TCP
  address = "tcp://questdb-target:9009"
```

A few things to note: * `omit_hostname` avoids an extra column. When monitoring multiple QuestDB instances, keep it enabled. * `url_tag` is set to blank for the same reason - by default the Prometheus plugin adds the URL as an extra column. * `metric_version = 2` ensures all metrics go into a single table, rather than one table per metric. * The `aggregators.merge` plugin rolls up metrics into a single row per data point (with multiple columns), rather than one row per metric. Without it, the table becomes very sparse. * The config uses a different hostname for the QuestDB output to collect metrics on a separate instance. This is recommended for production, but for development the same host can be used.

View this recipe online

Import CSV with millisecond timestamps

Import CSV files containing epoch timestamps in milliseconds into QuestDB.

Problem

QuestDB expects either date/timestamp literals, or epochs in microseconds or nanoseconds.

Solution options

Here are the options available:

Option 1: Pre-process the dataset Convert timestamps from milliseconds to microseconds before import. If importing lots of data, create Parquet files, copy them to the QuestDB import folder, and read them with `read_parquet('file.parquet')`. Then use `INSERT INTO SELECT` to copy to another table.

Option 2: Staging table Import into a non-partitioned table as DATE, then `INSERT INTO` a partitioned table as TIMESTAMP:

```
-- Create staging table
CREATE TABLE trades_staging (
  timestamp_ms LONG,
  symbol SYMBOL,
  price DOUBLE,
  amount DOUBLE
);

-- Import CSV to staging table (via web console or REST API)

-- Create final table
CREATE TABLE trades (
  timestamp TIMESTAMP,
  symbol SYMBOL INDEX,
  price DOUBLE,
  amount DOUBLE
) TIMESTAMP(timestamp) PARTITION BY DAY;

-- Convert and insert
INSERT INTO trades
SELECT
  cast(timestamp_ms * 1000 AS TIMESTAMP) as timestamp,
  symbol,
  price,
  amount
FROM trades_staging;

-- Drop staging table
DROP TABLE trades_staging;
```

You would be using twice the storage temporarily, but then you can drop the initial staging table.

Option 3: ILP client Read the CSV line-by-line and convert, then send via the ILP client.

View this recipe online

TLS with PgBouncer for QuestDB

Configure PgBouncer to provide TLS termination for QuestDB Open Source PostgreSQL wire protocol connections.

> **NOTE: QuestDB Enterprise**
>
> For QuestDB Enterprise, there is native TLS support, so you can connect directly with TLS or use PgBouncer with full TLS end-to-end encryption.

Solution: TLS termination at PgBouncer

QuestDB Open Source does not implement TLS on the PostgreSQL wire protocol, so TLS termination needs to be done at the PgBouncer level.

Configure PgBouncer with:

```
[databases]
questdb = host=127.0.0.1 port=8812 dbname=questdb user=admin password=quest

[pgbouncer]
listen_addr = 127.0.0.1
listen_port = 5432
auth_type = trust
auth_file = /path/to/pgbouncer/userlist.txt

client_tls_sslmode = require
client_tls_key_file = /path/to/pgbouncer/pgbouncer.key
client_tls_cert_file = /path/to/pgbouncer/pgbouncer.crt
client_tls_ca_file = /etc/ssl/cert.pem

server_tls_sslmode = disable
logfile = /path/to/pgbouncer/pgbouncer.log
pidfile = /path/to/pgbouncer/pgbouncer.pid
```

The key setting is `server_tls_sslmode = disable`. This makes psql connect using TLS to PgBouncer, but PgBouncer will connect without TLS to your QuestDB instance.

Connect with:

```
psql "host=127.0.0.1 port=5432 dbname=questdb user=admin sslmode=require"
```

> **WARNING: Unencrypted Traffic**
>
> Traffic will be unencrypted between PgBouncer and QuestDB. This setup is only suitable when both services run on the same host or within a trusted network.

View this recipe online

Copy data between QuestDB instances

Copy a subset of data from one QuestDB instance to another for testing or development purposes.

Problem

You want to copy data between QuestDB instances. This method allows you to copy any arbitrary query result, but if you want a full database copy please check the backup and restore documentation.

Solution: Table2Ilp utility

QuestDB ships with a `utils` folder that includes a tool to read from one instance (using the PostgreSQL protocol) and write into another (using ILP).

You would need to compile the jar, and then use it like this:

```
java -cp utils.jar io.questdb.cliutil.Table2Ilp \
  -d trades \
  -dilp "https::addr=localhost:9000;username=admin;password=quest;" \
  -s "trades WHERE start_time in '2022-06'" \
  -sc "jdbc:postgresql://localhost:8812/qdb?user=account&password=secret&ssl=false" \
  -sym "ticker,exchange" \
  -sts start_time
```

This reads from the source instance using PostgreSQL wire protocol and writes to the destination using ILP.

Alternative: Export endpoint

You can also use the export endpoint to export data to CSV or other formats.

View this recipe online

Query performance histogram

Create a histogram of query execution times using the `_query_trace` system table.

> **NOTE: Enable Query Tracing**
>
> Query tracing needs to be enabled for the `_query_trace` table to be populated.

Solution: Percentile-based histogram

We can create a subquery that first calculates the percentiles for each bucket, in this case at 10% intervals. Then on a second query we can do a `UNION` of 10 subqueries where each is doing a `CROSS JOIN` against the calculated percentiles and finding how many queries are below the threshold for the bucket.

Note in this case the histogram is cumulative, and each bucket includes the results from the smaller buckets as well. If we prefer non-cumulative, the condition would change from less than to `BETWEEN`.

```
WITH quantiles AS (
  SELECT
    approx_percentile(execution_micros, 0.10, 5) AS p10,
    approx_percentile(execution_micros, 0.20, 5) AS p20,
    approx_percentile(execution_micros, 0.30, 5) AS p30,
    approx_percentile(execution_micros, 0.40, 5) AS p40,
    approx_percentile(execution_micros, 0.50, 5) AS p50,
    approx_percentile(execution_micros, 0.60, 5) AS p60,
    approx_percentile(execution_micros, 0.70, 5) AS p70,
    approx_percentile(execution_micros, 0.80, 5) AS p80,
    approx_percentile(execution_micros, 0.90, 5) AS p90,
    approx_percentile(execution_micros, 1.0, 5)  AS p100
  FROM _query_trace
), cumulative_hist AS (
SELECT '10' AS bucket, p10 as micros_threshold, count(*) AS frequency
FROM _query_trace CROSS JOIN quantiles
WHERE execution_micros < p10

UNION ALL

SELECT '20', p20 as micros_threshold,  count(*)
FROM _query_trace CROSS JOIN quantiles
WHERE execution_micros < p20

UNION ALL

SELECT '30', p30 as micros_threshold, count(*)
FROM _query_trace CROSS JOIN quantiles
WHERE execution_micros < p30

UNION ALL

SELECT '40', p40 as micros_threshold, count(*)
FROM _query_trace CROSS JOIN quantiles
WHERE  execution_micros < p40

UNION ALL

SELECT '50', p50 as micros_threshold, count(*)
```

```
FROM _query_trace CROSS JOIN quantiles
WHERE  execution_micros < p50

UNION ALL

SELECT '60', p60 as micros_threshold, count(*)
FROM _query_trace CROSS JOIN quantiles
WHERE  execution_micros < p60

UNION ALL

SELECT '70', p70 as micros_threshold, count(*)
FROM _query_trace CROSS JOIN quantiles
WHERE  execution_micros < p70

UNION ALL

SELECT '80', p80 as micros_threshold, count(*)
FROM _query_trace CROSS JOIN quantiles
WHERE  execution_micros < p80

UNION ALL

SELECT '90', p90 as micros_threshold, count(*)
FROM _query_trace CROSS JOIN quantiles
WHERE  execution_micros < p90

UNION ALL

SELECT '100', p100 as micros_threshold, count(*)
FROM _query_trace CROSS JOIN quantiles
 )
 SELECT * FROM cumulative_hist;
```

Output:

```
"bucket","micros_threshold","frequency"
"10",215.0,26
"20",348.0,53
"30",591.0,80
"40",819.0,106
"50",1088.0,133
"60",1527.0,160
"70",2293.0,186
"80",4788.0,213
"90",23016.0,240
"100",1078759.0,267
```

View this recipe online

Optimize disk and memory usage with many tables

When operating QuestDB with many tables, the default settings may consume more memory and disk space than necessary. This recipe shows how to optimize these resources.

Problem

QuestDB allocates memory for out-of-order inserts per column and table. With the default setting of `cairo.o3.column.memory.size=256K`, each table and column uses 512K of memory (2x the configured size). When you have many tables, this memory overhead can become significant.

Similarly, QuestDB allocates disk space in chunks for columns and indexes. While larger chunks make sense for a single large table, multiple smaller tables benefit from smaller allocation sizes, which can noticeably decrease disk storage usage.

Solution

Reduce memory allocation for out-of-order inserts by setting a smaller `cairo.o3.column.memory.size`. Start with 128K and adjust based on your needs:

```
cairo.o3.column.memory.size=128K
```

Reduce disk space allocation by configuring smaller page sizes for data and indexes:

```
cairo.system.writer.data.append.page.size=128K
cairo.writer.data.append.page.size=128K
cairo.writer.data.index.key.append.page.size=128K
cairo.writer.data.index.value.append.page.size=128K
```

These settings should be added to your `server.conf` file or set as environment variables.
View this recipe online

Check transaction applied after ingestion

When ingesting data to a WAL table using ILP protocol, inserts are asynchronous. This recipe shows how to ensure all ingested rows are visible for read-only queries.

Problem

You're performing a single-time ingestion of a large data volume using ILP protocol to a table that uses Write-Ahead Log (WAL). Since inserts are asynchronous, you need to confirm that all ingested rows are visible for read-only queries before proceeding with operations.

Solution

Query the `wal_tables()` function to check if the writer transaction matches the sequencer transaction. When these values match, all rows have become visible:

Check applied transactions from WAL files

```
SELECT *
FROM wal_tables()
WHERE name = 'core_price' AND writerTxn = sequencerTxn;
```

Run on demo.questdb.com

This query returns a row when `writerTxn` equals `sequencerTxn` for your table: - `writerTxn` is the last committed transaction available for read-only queries - `sequencerTxn` is the last transaction committed to WAL

When they match, all WAL transactions have been applied and all rows are visible for queries.

Another viable approach is to run `SELECT count(*) FROM my_table` and verify the expected row count.

View this recipe online

Show parameters with non-default values

When troubleshooting or auditing your QuestDB configuration, it's useful to see which parameters have been changed from their defaults.

Problem

You need to identify which configuration parameters have been explicitly set via the configuration file or environment variables, filtering out all parameters that are still using their default values.

Solution

Query the `SHOW PARAMETERS` command and filter by `value_source` to exclude defaults:

Find which params where modified from default values

```
-- Show all parameters modified from their defaults, via conf file or env variable
(SHOW PARAMETERS) WHERE value_source <> 'default';
```

Run on demo.questdb.com

This query returns only the parameters that have been explicitly configured, showing their current values and the source of the configuration (e.g., `conf` file or `env` variable).

View this recipe online

Appendix

Demo Data Schema

The QuestDB demo instance at demo.questdb.com contains two datasets that you can query directly: simulated FX market data and real cryptocurrency trades. This page describes the available tables and their structure.

> **TIP: The demo instance is read-only. For testing write operations (INSERT, UPDATE, DELETE), you'll need to run QuestDB locally. See the Quick Start guide for installation instructions.**

Overview

The demo instance provides two independent datasets:

1. **FX Market Data (Simulated)** - Foreign exchange prices and order books
2. **Cryptocurrency Trades (Real)** - Live cryptocurrency trades from OKX exchange

---

FX market data (simulated)

The FX dataset contains simulated foreign exchange market data for 30 currency pairs. We fetch real reference prices from Yahoo Finance every few seconds, but all order book levels and price updates are generated algorithmically based on these reference prices.

core_price table The `core_price` table contains individual FX price updates from various liquidity providers. Each row represents a bid/ask quote update for a specific currency pair from a specific ECN.

Schema `sql title="core_price table structure" CREATE TABLE 'core_price' ( timestamp TIMESTAMP, symbol SYMBOL, ecn SYMBOL, bid_price DOUBLE, bid_volume LONG, ask_price DOUBLE, ask_volume LONG, reason SYMBOL, indicator1 DOUBLE, indicator2 DOUBLE ) timestamp(timestamp) PARTITION BY HOUR TTL 3 DAYS;`

Columns

- **`timestamp`** - Time of the price update (designated timestamp)
- **`symbol`** - Currency pair from the 30 tracked symbols (see list below)
- **`ecn`** - Electronic Communication Network providing the quote: **LMAX**, **EBS**, **Currenex**, or **Hotspot**
- **`bid_price`** - Bid price (price at which market makers are willing to buy)
- **`bid_volume`** - Volume available at the bid price
- **`ask_price`** - Ask price (price at which market makers are willing to sell)
- **`ask_volume`** - Volume available at the ask price
- **`reason`** - Reason for the price update: "normal", "liquidity_event", or "news_event"
- **`indicator1`**, **`indicator2`** - Additional market indicators

The table tracks **30 currency pairs**: EURUSD, GBPUSD, USDJPY, USDCHF, AUDUSD, USDCAD, NZDUSD, EURJPY, GBPJPY, EURGBP, AUDJPY, CADJPY, NZDJPY, EURAUD, EURNZD, AUDNZD, GBPAUD, GBPNZD, AUDCAD, NZDCAD, EURCAD, EURCHF, GBPCHF, USDNOK, USDSEK, USDZAR, USDMXN, USDSGD, USDHKD, USDTRY.

Sample data **Recent core_price updates**

```
SELECT * FROM core_price
WHERE timestamp IN today()
LIMIT -10;
```

Run on demo.questdb.com

Results:

timestamp	symbol	bidcn	bid_price	bid_volume	ask_price	ask_volume	reason	indicator1	indicator2
2025-12-18T11:46:13.059566Z	USDCHF	MAX	0.7959	219884	0.7971	223174	liquidity_event	0.641	
2025-12-18T11:46:13.060542Z	USDSGD	Currenex	1.291	29575704	1.2982	30121560	normal	0.034	
2025-12-18T11:46:13.061853Z	EURAUD	MAX	1.7651	6207630	1.7691	5631029	liquidity_event	0.027	
2025-12-18T11:46:13.064138Z	AUDNZD	MAX	1.1344	227668	1.1356	212604	liquidity_event	0.881	
2025-12-18T11:46:13.065041Z	GBPNZD	MAX	2.3307	2021166	2.3337	1712096	normal	0.308	
2025-12-18T11:46:13.065187Z	USDCAD	EBS	1.3837	2394978	1.3869	2300556	normal	0.084	
2025-12-18T11:46:13.065722Z	USDZAR	EBS	16.721	1281070	16.726	3235365	liquidity_event	0.151	
2025-12-18T11:46:13.066128Z	EURAUD	EBS	1.763	8104718	1.7712	8834247	news_event	0.027	
2025-12-18T11:46:13.066700Z	CADJPY	Currenex	113.63	2030082	114.11	1972091	normal	0.55	
2025-12-18T11:46:13.071607Z	NZDJPY	Currenex	89.95	3528422	90.46	3055252	liquidity_event	0.69	

market_data table The `market_data` table contains order book snapshots for currency pairs. Each row represents a complete view of the order book at a specific timestamp, with bid and ask prices and volumes stored as 2D arrays.

Schema `sql title="market_data table structure" CREATE TABLE 'market_data' ( timestamp TIMESTAMP, symbol SYMBOL CAPACITY 16384 CACHE, bids DOUBLE[][], asks DOUBLE[][] ) timestamp(timestamp) PARTITION BY HOUR TTL 3 DAYS;`

Columns

- **`timestamp`** - Time of the order book snapshot (designated timestamp)
- **`symbol`** - Currency pair (e.g., EURUSD, GBPJPY)
- **`bids`** - 2D array containing bid prices and volumes: `[[price1, price2, ...], [volume1, volume2, ...]]`
- **`asks`** - 2D array containing ask prices and volumes: `[[price1, price2, ...], [volume1, volume2, ...]]`

The arrays are structured so that: - `bids[1]` contains bid prices (descending order - highest first) - `bids[2]` contains corresponding bid volumes - `asks[1]` contains ask prices (ascending order - lowest first) - `asks[2]` contains corresponding ask volumes

Sample query Recent order book snapshots

```
SELECT timestamp, symbol,
       array_count(bids[1]) as bid_levels,
       array_count(asks[1]) as ask_levels
FROM market_data
WHERE timestamp IN today()
LIMIT -5;
```

Run on demo.questdb.com
Results:

timestamp	symbol	bid_levels	ask_levels
2025-12-18T12:04:07.071512Z	EURAUD	40	40
2025-12-18T12:04:07.072060Z	USDJPY	40	40
2025-12-18T12:04:07.072554Z	USDMXN	40	40
2025-12-18T12:04:07.072949Z	USDCAD	40	40
2025-12-18T12:04:07.073002Z	USDSEK	40	40

Each order book snapshot contains 40 bid levels and 40 ask levels.

fx_trades table The `fx_trades` table contains simulated FX trade executions. Each row represents a trade that executed against the order book, with realistic partial fills and level walking.

Schema `sql title="fx_trades table structure" CREATE TABLE 'fx_trades' ( timestamp TIMESTAMP_NS, symbol SYMBOL, ecn SYMBOL, trade_id UUID, side SYMBOL, passive BOOLEAN, price DOUBLE, quantity DOUBLE, counterparty SYMBOL, order_id UUID ) timestamp(timestamp) PARTITION BY HOUR TTL 1 MONTH;`

Columns

- **`timestamp`** - Time of trade execution with nanosecond precision (designated timestamp)
- **`symbol`** - Currency pair (same 30 pairs as `core_price`)
- **`ecn`** - ECN where trade executed: **LMAX**, **EBS**, **Currenex**, or **Hotspot**
- **`trade_id`** - Unique identifier for this specific trade
- **`side`** - Trade direction: **buy** or **sell**
- **`passive`** - Whether this was a passive (limit) or aggressive (market) order
- **`price`** - Execution price
- **`quantity`** - Trade size
- **`counterparty`** - 20-character LEI (Legal Entity Identifier) of the counterparty
- **`order_id`** - Parent order identifier (multiple trades can share the same `order_id` for partial fills)

Sample data Recent FX trades

```
SELECT * FROM fx_trades
WHERE timestamp IN today()
LIMIT -10;
```

Run on demo.questdb.com
Results:

timestamp	symbol	ecn	trade_id	side	passive	price	quantity	counterparty	order_id
2026-01-12T12:18:57.138282586Z	EURUSD	LMAX	d14e6e54-6c6b-495d-865d-47311a36519b	sell	false	1.1705	5936100	004409EA0	6B5B3FE6954B-c3e7-4909-8a64-31a2b6f0f9c0
2026-01-12T12:18:57.138912209Z	EURUSD	LMAX	be857ed7-848f-4d23-83ff-3e5636cbc9de	sell	false	1.1707	7077400	000A4FB27	6B3B3E86F143-c3e7-4909-8a64-31a2b6f0f9c0
2026-01-12T12:18:57.259555330Z	GBPUSD	EBS	446cac16-9b25-4205-b1e1-3eda4a3bb539	sell	false	1.3401	9270000	0119FEF9	8D9E4387-9D158929-4c48-bc18-7164b1a956e3
2026-01-12T12:18:57.303333947Z	GBPUSD	EBS	27515a12-9ab6-4175-8fa3-422d4529f365	sell	true	1.3406	4629500	0363EC84	8B9C0598-D36C-fc45-4e1c-bd45-8933909a67fc
2026-01-12T12:18:57.334406432Z	USDTRY	EBS	c82453b3-9961-40ea-a6ac-43c33fe0f235	sell	true	43.1065	8849000	02A80CCE	24AFD373-90642-0f21-4241-8ca7-903080848dc0
2026-01-12T12:18:57.365445776Z	USDJPY	LMAX	bf918a88-60c2-4a20-8f53-65298b5a10fe	buy	false	156.82	5554800	0EB428CC	7C5B5C240F71-65fa-4ffb-8fa8-840e88d2c316
2026-01-12T12:18:57.479674129Z	USDJPY	EBS	c7c902bd-7075-4952-88d1-76d39ba4c706	buy	false	156.82	9859100	0A10D276	5982296A0161-684f-4783-9e8c-7206519a85f8
2026-01-12T12:18:57.480051522Z	USDJPY	EBS	a20b6f91-7148-4b64-8a36-85da5bec66f9	buy	false	156.85	7815200	0CBD8496	5952846C8554-684f-4783-9e8c-7206519a85f8
2026-01-12T12:18:57.509773474Z	GBPUSD	Cbreme	ec6b771b-5abd-44c7-9e0e-3527ce6fb5b4	sell	false	1.3406	4230500	06728CF2	54EB4941-2B98-4a5c-8b67-d5cd961ec5e8
2026-01-12T12:18:57.334732460Z	USDTRY	EBS	469637a5-6553-4aad-aad9-f7114c8a442d	sell	true	43.1	1011700	02CAC92	23A7B4B3-D30C-0f21-4241-8ca7-903080848dc0

FX materialized views The FX dataset includes several materialized views providing pre-aggregated data at different time intervals:

Best bid/offer (BBO) views

- `bbo_1s` - Best bid and offer aggregated every 1 second
- `bbo_1m` - Best bid and offer aggregated every 1 minute
- `bbo_1h` - Best bid and offer aggregated every 1 hour

- **`bbo_1d`** - Best bid and offer aggregated every 1 day

Core price aggregations

- **`core_price_1s`** - Core prices aggregated every 1 second
- **`core_price_1d`** - Core prices aggregated every 1 day

Market data OHLC

- **`market_data_ohlc_1m`** - Open, High, Low, Close candlesticks at 1-minute intervals
- **`market_data_ohlc_15m`** - OHLC candlesticks at 15-minute intervals
- **`market_data_ohlc_1d`** - OHLC candlesticks at 1-day intervals

FX trades OHLC

- **`fx_trades_ohlc_1m`** - OHLC candlesticks from trade executions at 1-minute intervals
- **`fx_trades_ohlc_1h`** - OHLC candlesticks from trade executions at 1-hour intervals

These views are continuously updated and optimized for dashboard and analytics queries on FX data.

FX data volume

- **`market_data`**: Approximately **160 million rows** per day (order book snapshots)
- **`core_price`**: Approximately **73 million rows** per day (price updates across all ECNs and symbols)
- **`fx_trades`**: Approximately **5.1 million rows** per day (trade executions)

Cryptocurrency trades (real)

The cryptocurrency dataset contains **real market data** streamed live from the OKX exchange using FeedHandler. These are actual executed trades, not simulated data.

trades table The `trades` table contains real cryptocurrency trade data. Each row represents an actual executed trade for a cryptocurrency pair.

Schema `sql title="trades table structure" CREATE TABLE 'trades' ( symbol SYMBOL CAPACITY 256 CACHE, side SYMBOL CAPACITY 256 CACHE, price DOUBLE, amount DOUBLE, timestamp TIMESTAMP ) timestamp(timestamp) PARTITION BY DAY;`

Columns

- **`timestamp`** - Time when the trade was executed (designated timestamp)
- **`symbol`** - Cryptocurrency trading pair from the 12 tracked symbols (see list below)
- **`side`** - Trade side: **buy** or **sell**
- **`price`** - Execution price of the trade
- **`amount`** - Trade size (volume in base currency)

The table tracks **12 cryptocurrency pairs**: ADA-USDT, AVAX-USDT, BTC-USDT, DAI-USDT, DOT-USDT, ETH-BTC, ETH-USDT, LTC-USDT, SOL-BTC, SOL-USDT, UNI-USDT, XLM-USDT.

Sample data **Recent cryptocurrency trades**

```
SELECT * FROM trades
LIMIT -10;
```

Run on demo.questdb.com
Results:

symbol	side	price	amount	timestamp
BTC-USDT	buy	85721.6	0.00045714	2025-12-18T19:31:11.203000Z
BTC-USDT	buy	85721.6	0.00045714	2025-12-18T19:31:11.203000Z
BTC-USDT	buy	85726.6	0.00001501	2025-12-18T19:31:11.206000Z
BTC-USDT	buy	85726.6	0.00001501	2025-12-18T19:31:11.206000Z
BTC-USDT	buy	85726.9	0.000887	2025-12-18T19:31:11.206000Z
BTC-USDT	buy	85726.9	0.000887	2025-12-18T19:31:11.206000Z
BTC-USDT	buy	85731.3	0.00004393	2025-12-18T19:31:11.206000Z
BTC-USDT	buy	85731.3	0.00004393	2025-12-18T19:31:11.206000Z
ETH-USDT	sell	2827.54	0.006929	2025-12-18T19:31:11.595000Z
ETH-USDT	sell	2827.54	0.006929	2025-12-18T19:31:11.595000Z

Cryptocurrency materialized views The cryptocurrency dataset includes materialized views for aggregated trade data:

Trades aggregations

- `trades_latest_1d` - Latest trade data aggregated daily
- `trades_OHLC_15m` - OHLC candlesticks for cryptocurrency trades at 15-minute intervals

These views are continuously updated and provide faster query performance for cryptocurrency trade analysis.

Cryptocurrency data volume

- `trades`: Approximately **3.7 million rows** per day (real cryptocurrency trades)

Data retention

FX tables (`core_price` and `market_data`) use a **3-day TTL (Time To Live)**, meaning data older than 3 days is automatically removed. This keeps the demo instance responsive while providing sufficient recent data.

Cryptocurrency trades table has **no retention policy** and contains historical data dating back to **March 8, 2022**. This provides over 3 years of real cryptocurrency trade history for long-term analysis and backtesting.

Using the demo data

You can run queries against both datasets directly on demo.questdb.com. Throughout the Cookbook, recipes using demo data will include a direct link to execute the query.

See the schema page online